

UNIT-3

①

Red-Black Trees, Splay Trees, Applications.

Hash Tables: Introduction, Hash structure, Hash function, Linear open addressing, chaining and Applications.

Red-Black tree :-

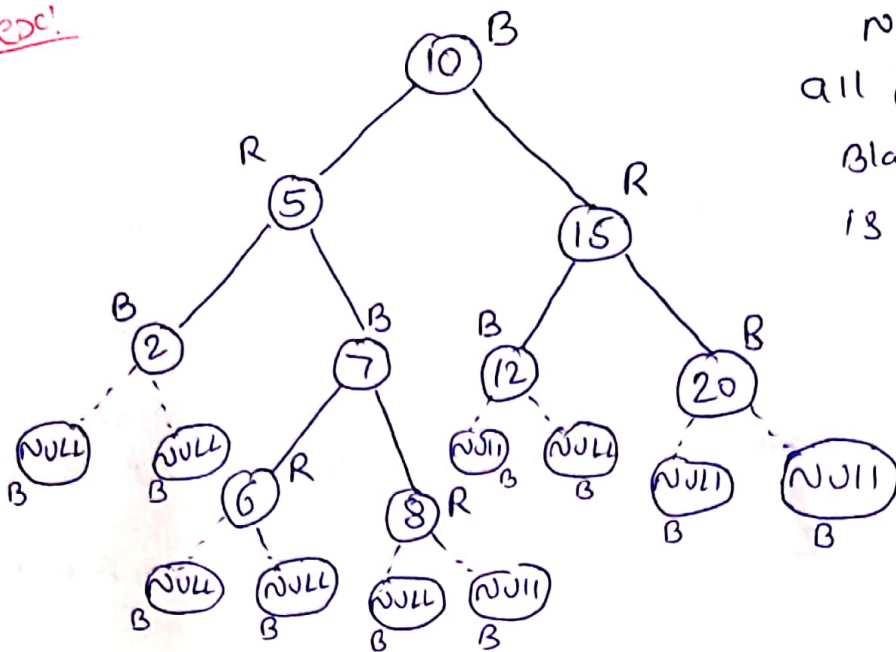
Red-Black tree is another variant of BST in which every node is colored either RED or BLACK. we can

In Red-Black tree, the color of a node is decided based on the properties of Red-Black tree. -

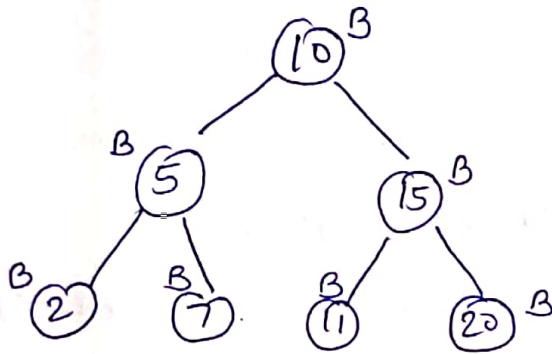
Properties of Red-Black tree :-

- ① Red-Black tree must be a BST
- ② The root node must be colored BLACK
- ③ The children of Red colored node must be colored Black. (There should not be two consecutive Red nodes)
- ④ In all the paths of the tree, there should be same number of Black colored nodes.
- ⑤ Every new node must be inserted with Red color
- ⑥ Every leaf (i.e. NULL node) must be colored BLACK

NOTE: Every Red Black tree is a BST but every BST need not be Red-Black tree.

Ex:

now this tree satisfies all the properties of Red-Black tree. so, this tree is a Red-Black tree.

Ex:

This tree is also a Red-Black tree. Even no Red node is there it follows all properties of Red-Black tree.

Note: A Perfect Binary tree is a Red-Black-Tree which contains only Black nodes

Note:- Every AVL Tree is a Red Black tree BUT every Red-Black Tree is not a AVL Tree.

Operations:

- Searching
- Insertion
- Deletion

3

The search tree operation is similar like BST.

Insertion:- Algorithm:

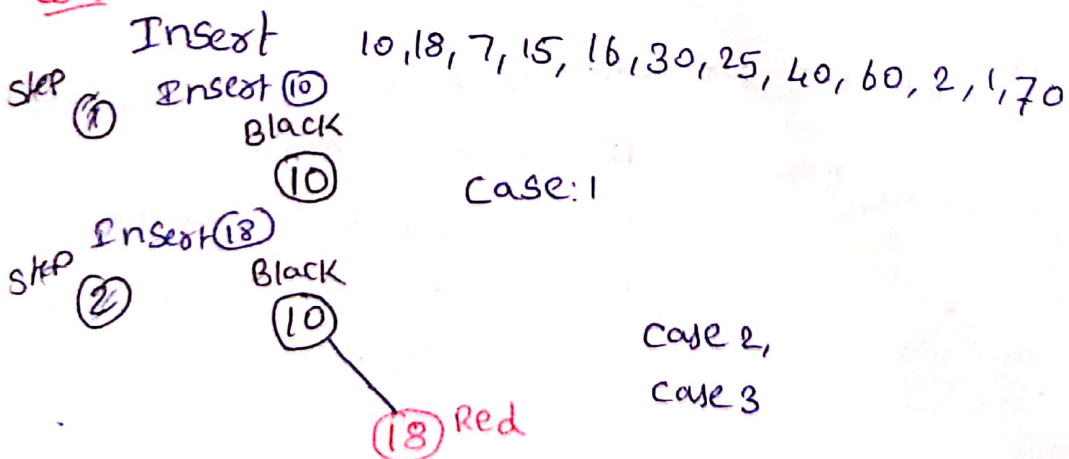
Cases:

- ① If tree is empty, create new-node as root node with color Black
- ② If tree is not empty. create new-node as leaf-node with color Red
- ③ If Parent of new-node is black then exit
- ④ If Parent of new-node is Red, then check the color of Parent's sibling of new node.

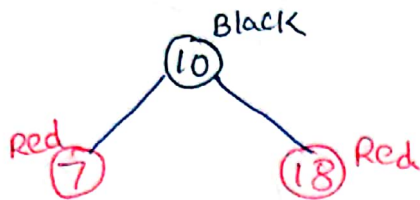
a) If color of Parent's sibling of new node is black then do suitable rotation & recolor

b) If color of Parent's sibling of new node is Red then recolor & also check if Parent's Parent of new-node is not root then recolor it & check

ex:-



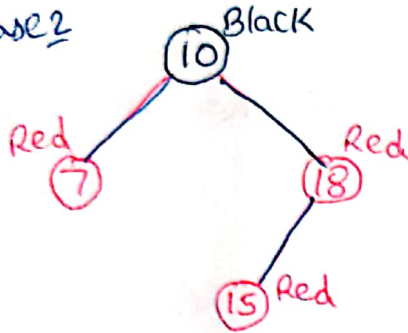
Step ③ Insert ⑦



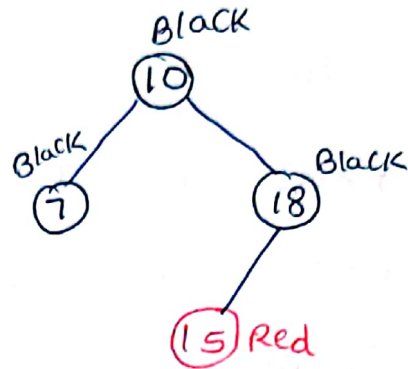
case 2, 3
case 3

Insert ⑮

case 2



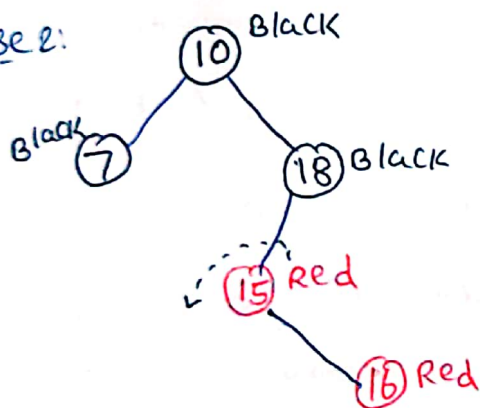
Apply
Case 4(b)



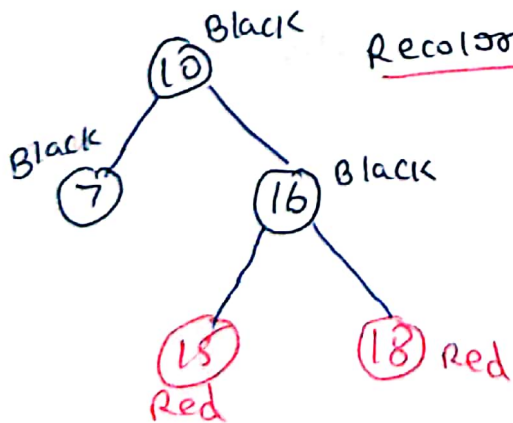
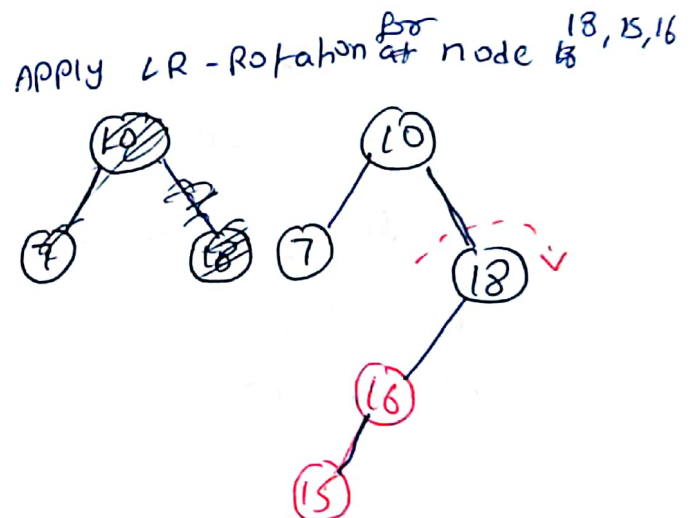
According to case 4(b)
check Parents
~~check Parents~~
Change the color of Parent and Parent sibling of new node and Check grand parent is root or not if root exit

Insert ⑮

case 2:



Apply
case 4(a)



Recolor for nodes 18 & 16

18 → Black to Red

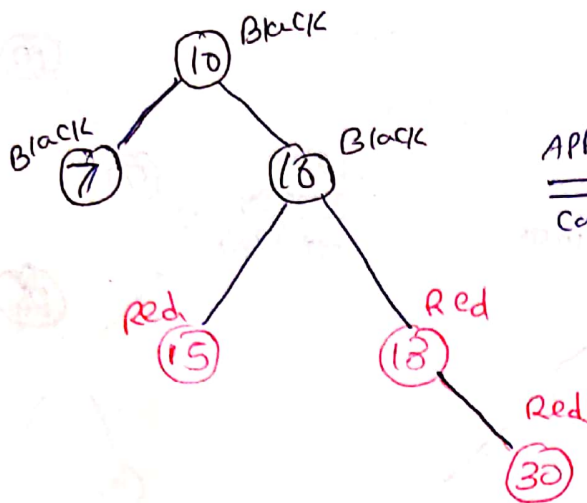
16 → Red to Black

15 → will remain in same place

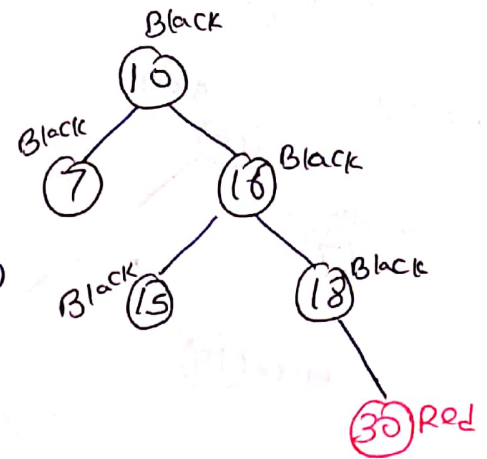
So, no change in color

Insert 30

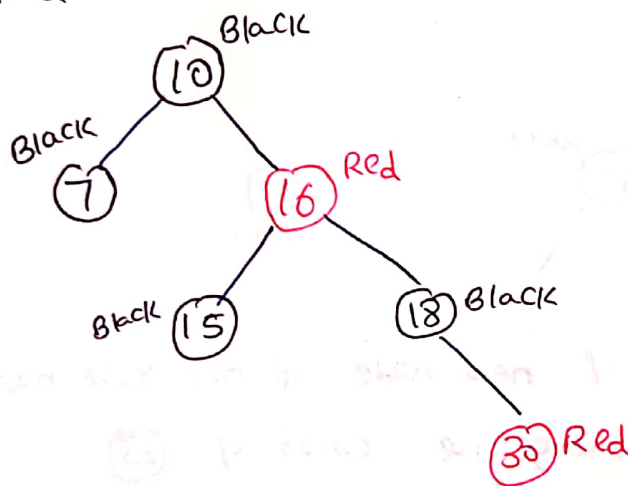
Case - 2



APPLY
Case 4(b)

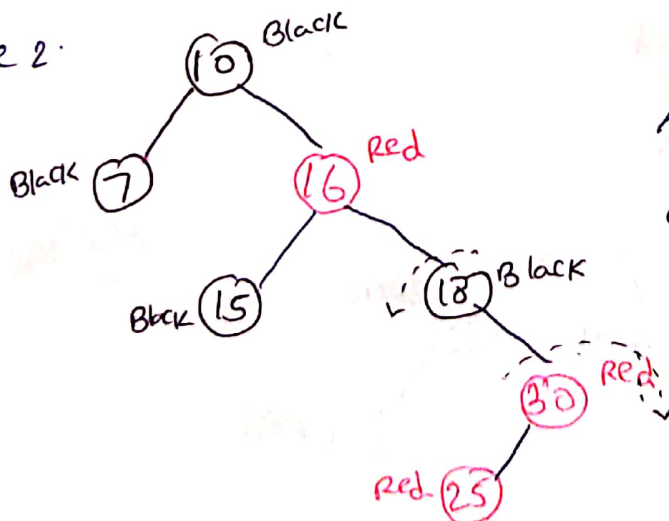


According to case 4(b) After changing the color of Parent and Parent's Sibling of new node check Grand Parent of new node if it is not root recolor, so 16 will become Red



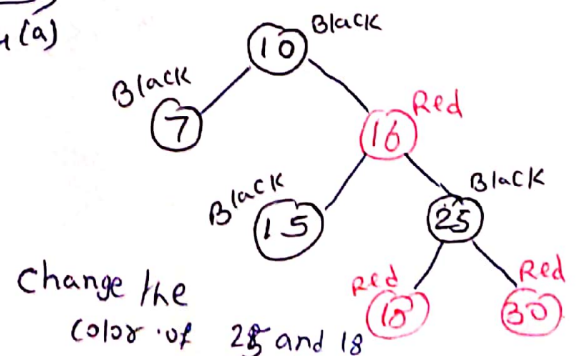
Insert 25

Case 2



APPLY
Case 4(a)

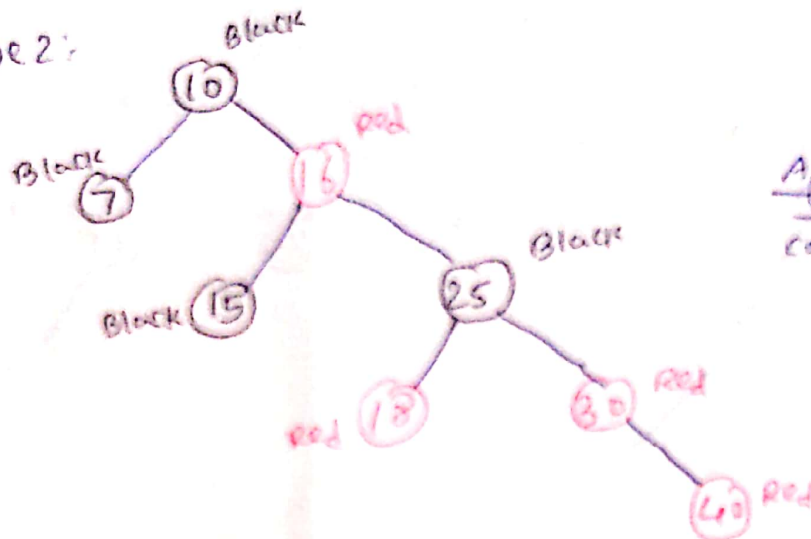
Do RL Rotation for nodes 18, 30, 25 and recolor



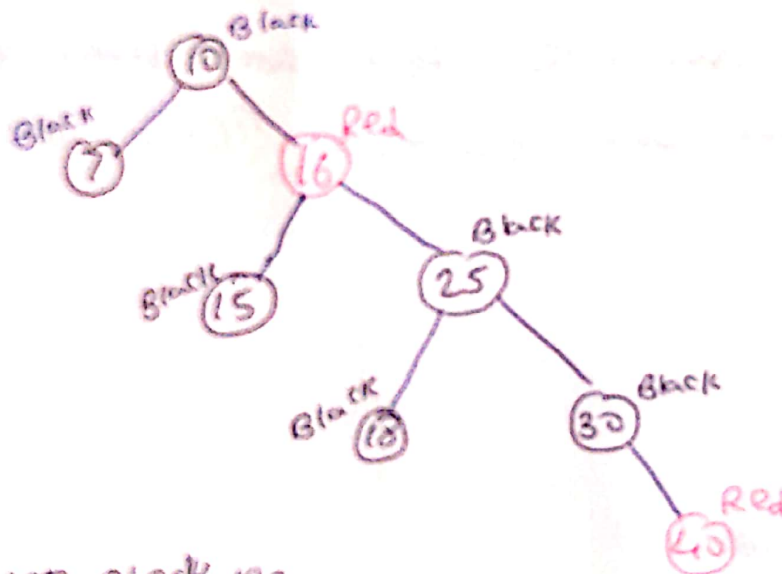
Change the color of 25 and 18

Insert 40

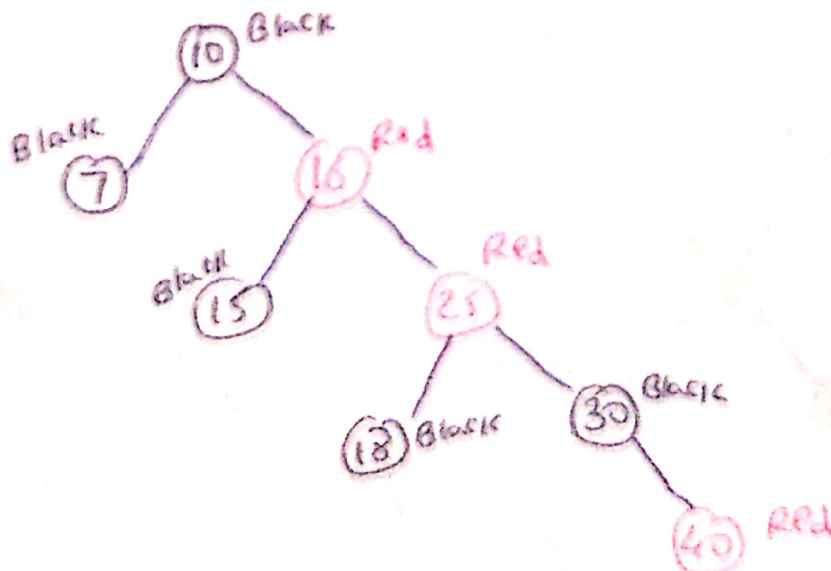
Case 2:



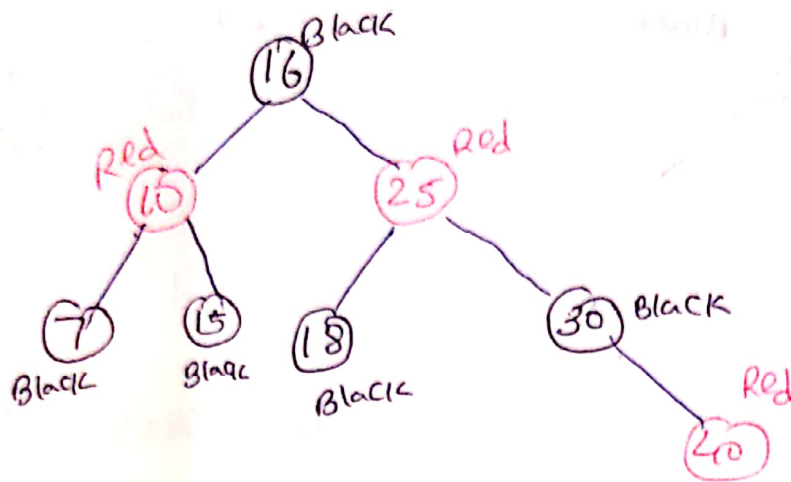
Apply Case 4(b)



then check the grand parent of new node if not root node then rotate and check. so, change the color of 25

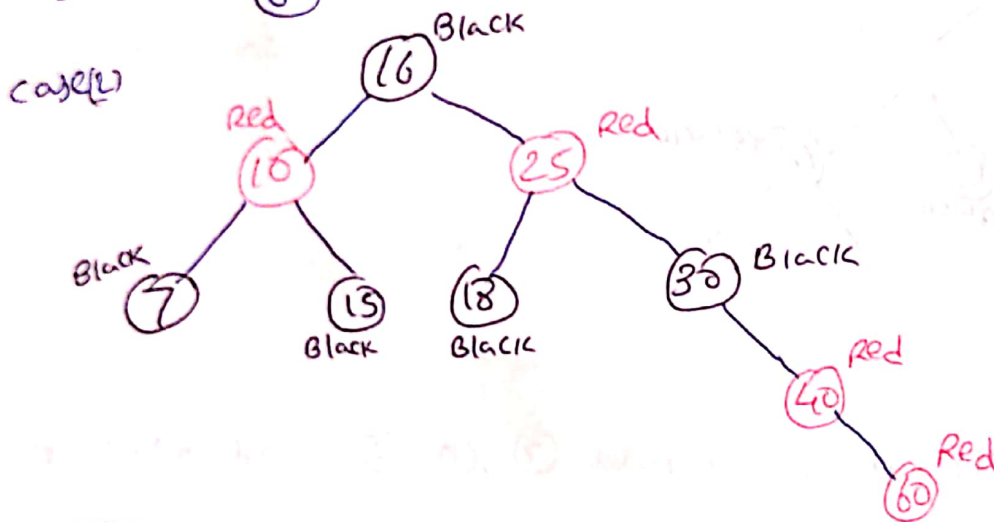


Now Red-Red conflict is there if we recheck so again apply case 4(a) Assuming (25) is new child now rotate at node 10, 16, 25



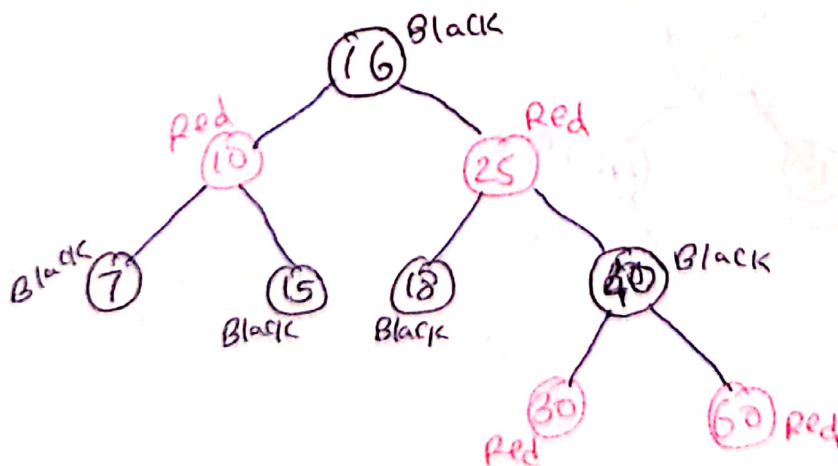
change the color of (16) and (10)

Insert (60)



Apply case 4(a).

make rotation at (30, 40, 60) and change the color of 30, 40



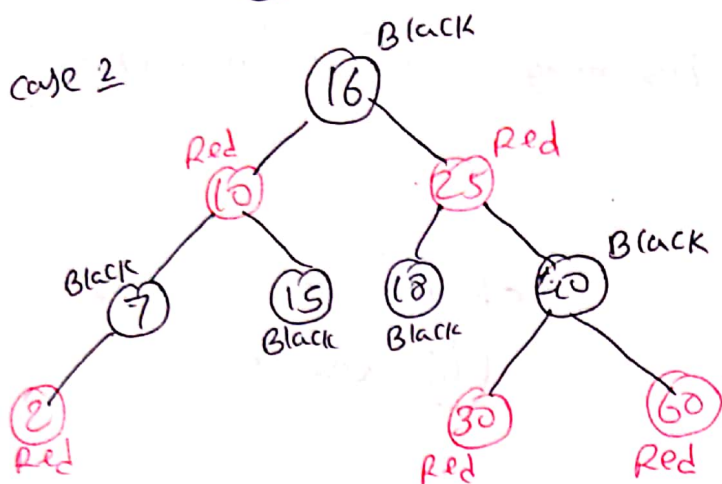
Insert ②

case 2

According to

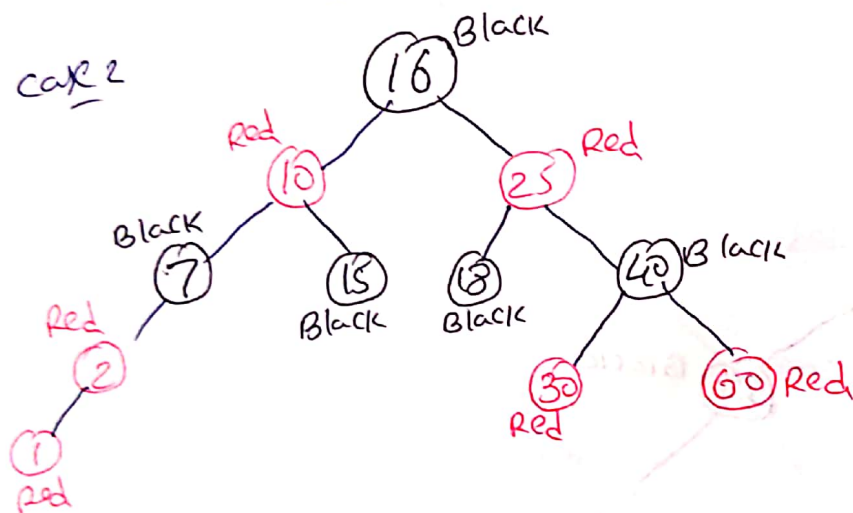
Case 3) Parent of

new node is black
so exit.



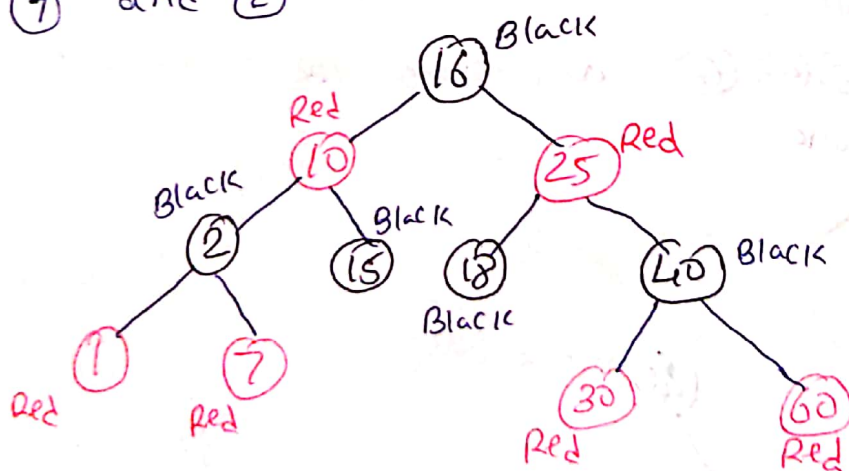
Insert ①

case 2



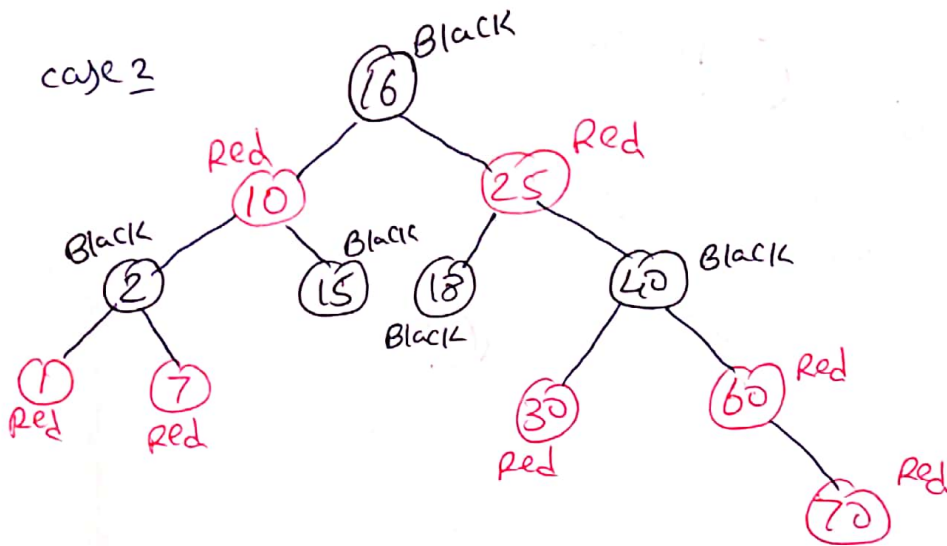
Apply case 4(a) rotate at nodes ⑦, ②, ① and recolor

⑦ and ②

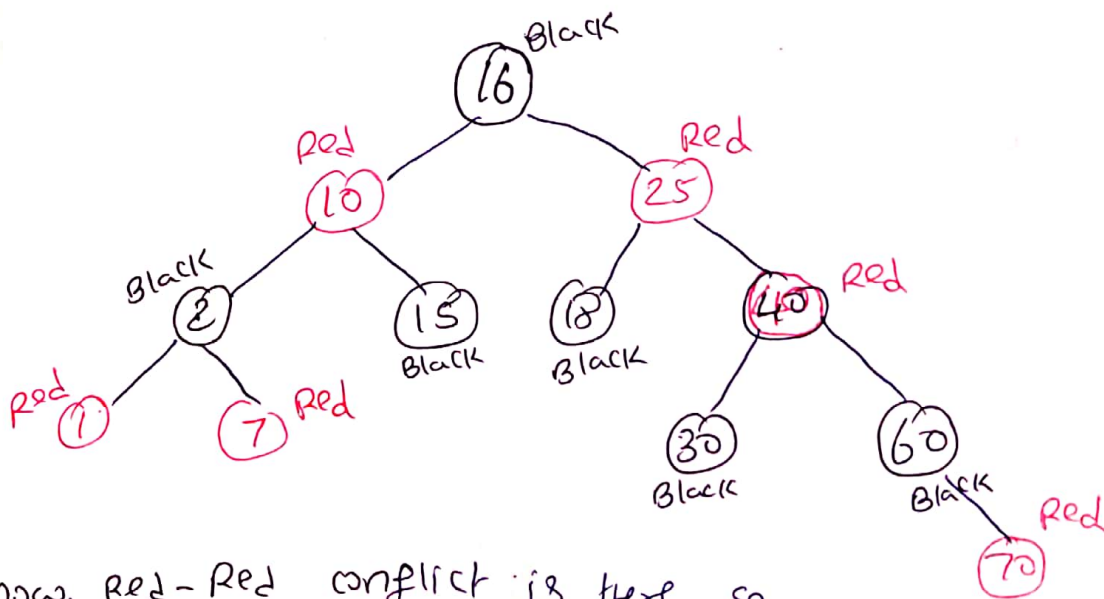


Insert (70)

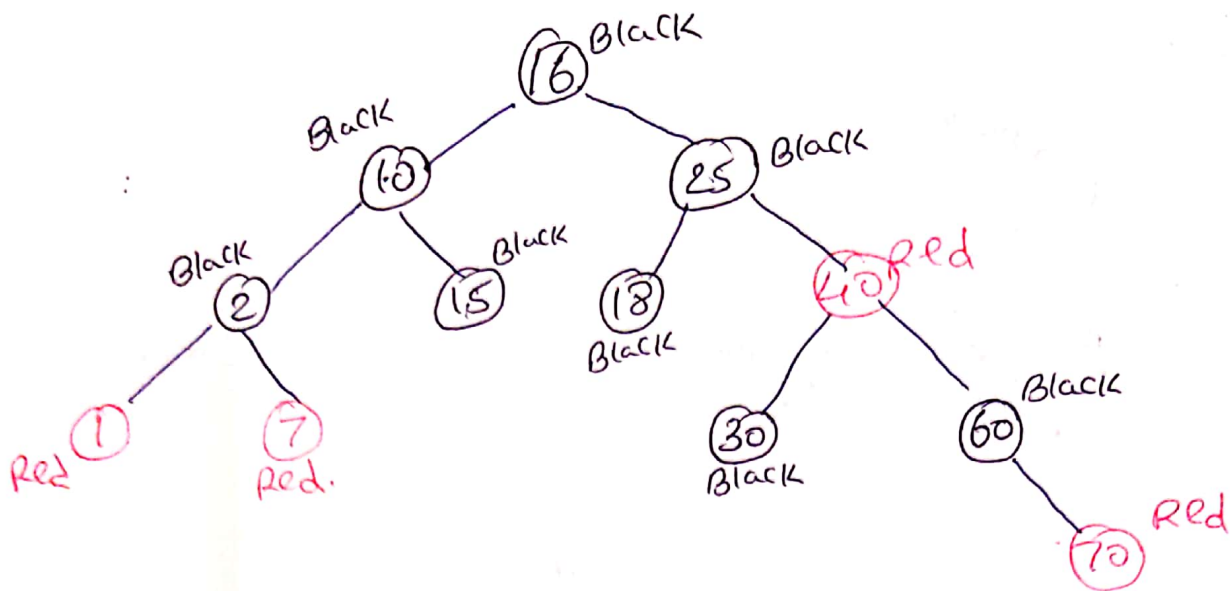
case 2



APPLY case 4(b) change the colors of 60, & 30 and check Grand Parent node. It is not root, so check Grand Parent node and recheck.



Now, Red-Red conflict is there so, Assume 40, is new child APPLY case 4(b) at (25). Change the colors of 25, & 10 which are parent & parent sibling of new node i.e. (40) and check Grand Parent node is root. if it is root exit.



Final Tree, Satisfies Red-Black Properties.

Note:-

check, After each insertion whether ^{Tree} it satisfies Red-Black Property or not. If not satisfying then recheck, rotate and rotate according to cases to make tree into Red-Black tree.

Deletion in Red-Black Tree:-

Algorithm

- ① perform BST deletion
- ② case 1:- If node to be deleted is Red just delete it
- case 2:- If root is Double Black (DB), just remove DB
- Case 3:- If DB's sibling is black, and both its children are black, then,
 - remove DB and Add Black to its parent
 - case 3.1:- if Parent is Red it becomes Black

(11)

Case 3.2:- if Parent is black it becomes double Black(DB)

- and make sibling red in either of cases 3.1 & 3.2
- check if still DB exist, then apply other cases.

Case 4:- If PB's sibling is red then

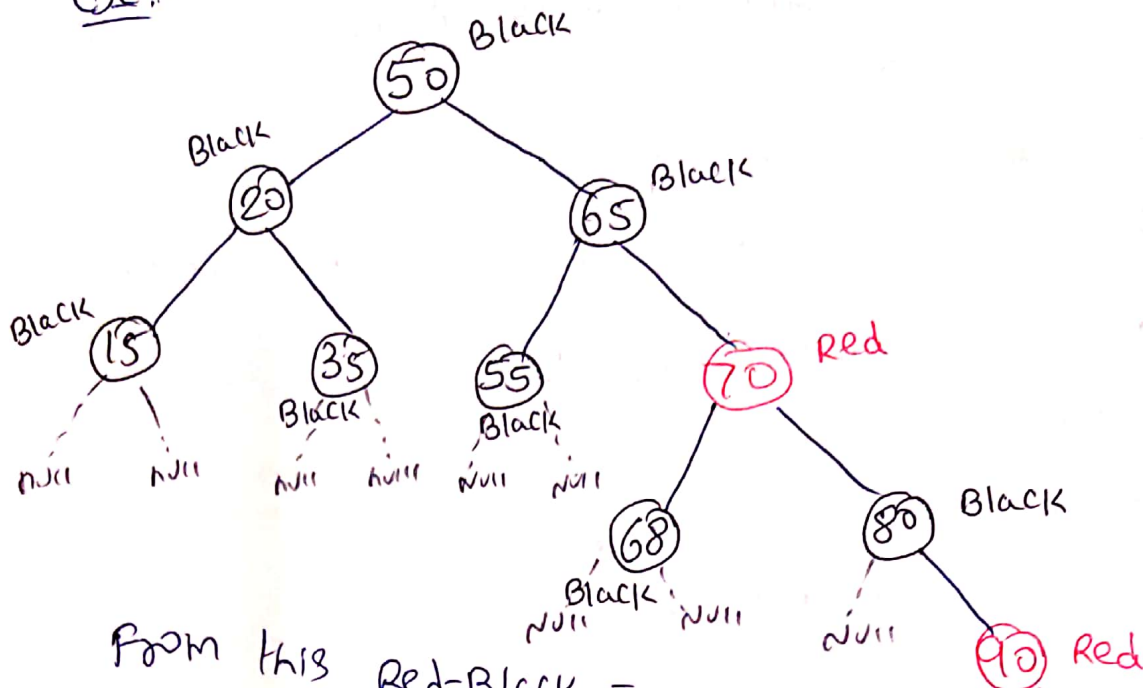
- swap colors of Parent and its sibling
- rotate Parent in DB direction.
- reapply cases, until DB is removed

Case 5:- If DB's sibling is Black, and sibling child who is far from DB is black, but near child of DB is Red then

- swap color of PB's sibling and sibling child who is near to DB
- rotate sibling in opposite direction to DB
- reapply cases until DB is removed

Case 6:- If DB's sibling is black, and far child is red then

- swap color of Parent and sibling
- rotate Parent in DB's direction
- remove DB
- change the color of red child to black.



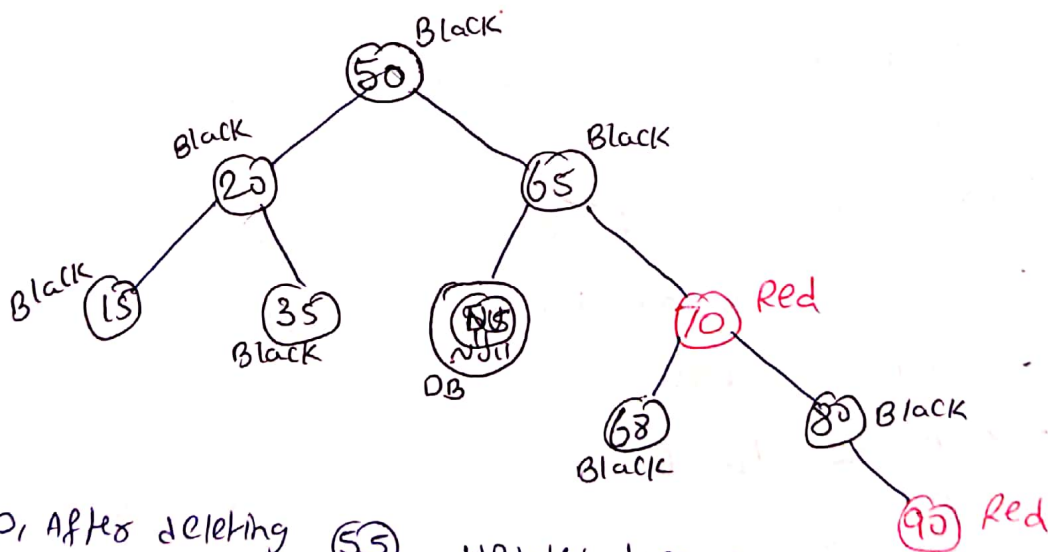
From this Red-Black Tree

Delete

nodes 55, 20, 90, 80,

50, 35, 15, 65, 68.
Delete (55)

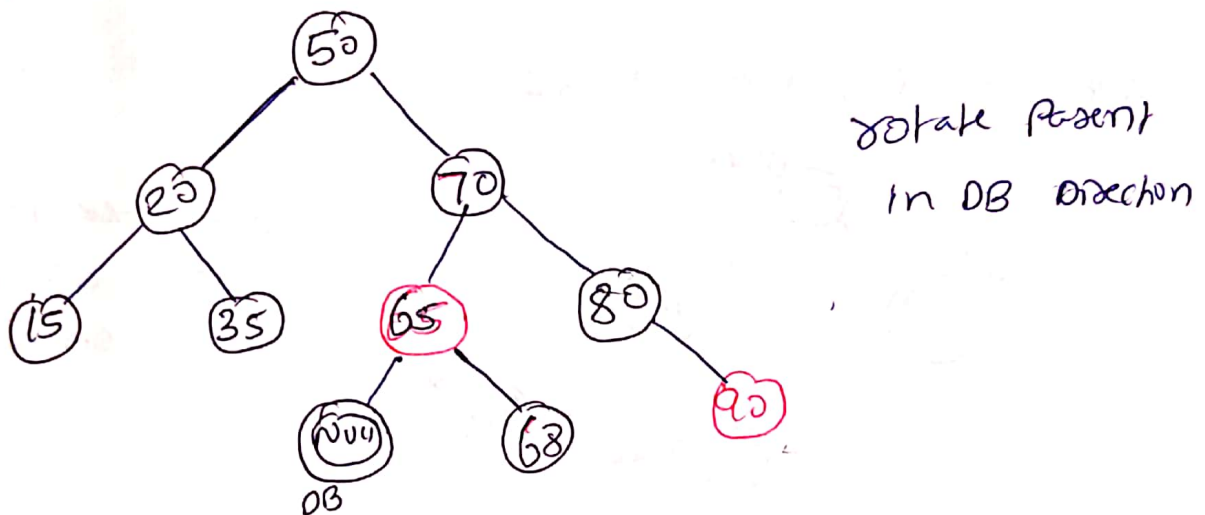
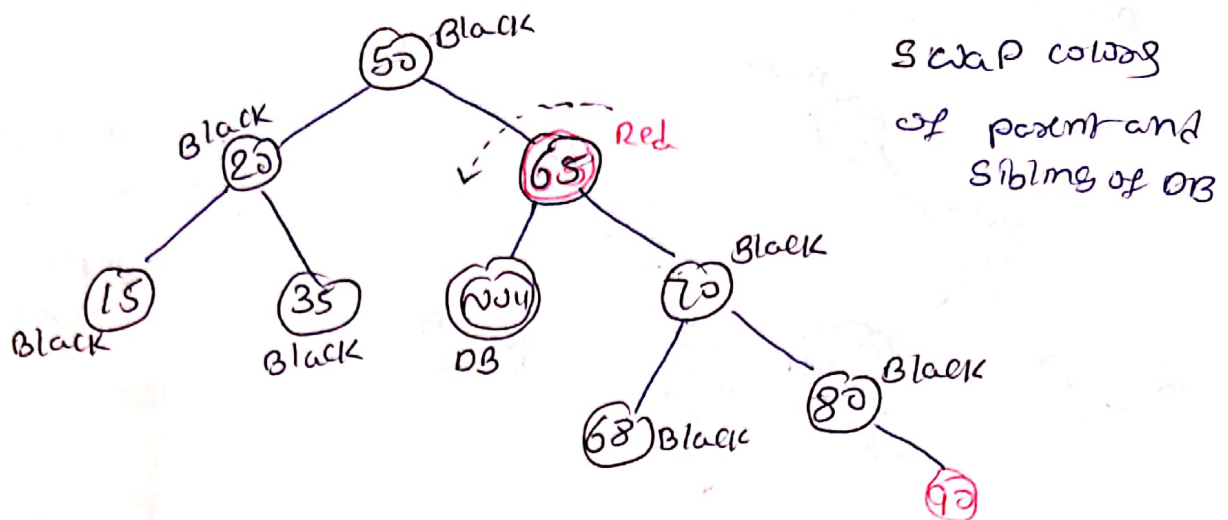
As per BST check search for the node (55). since it is a leaf node as per BST delete it directly but we have null node which is Black so replace NULL in the place of 55. so, 55 will become null node with double Black



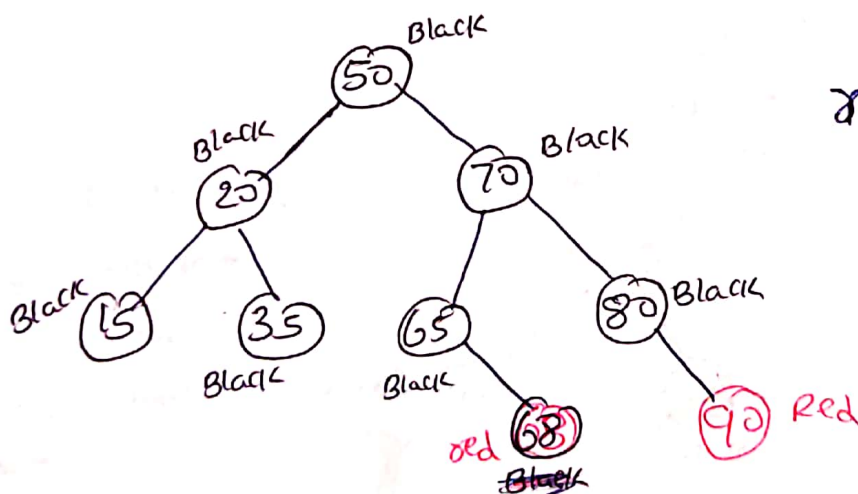
So, After deleting (55) updated tree is as above but we had here Double Black. we need to remove DB and tree should satisfy Red-Black Tree

(13)

According to case (14) the updated tree is as follows



now again apply case 3) and case (3-1)



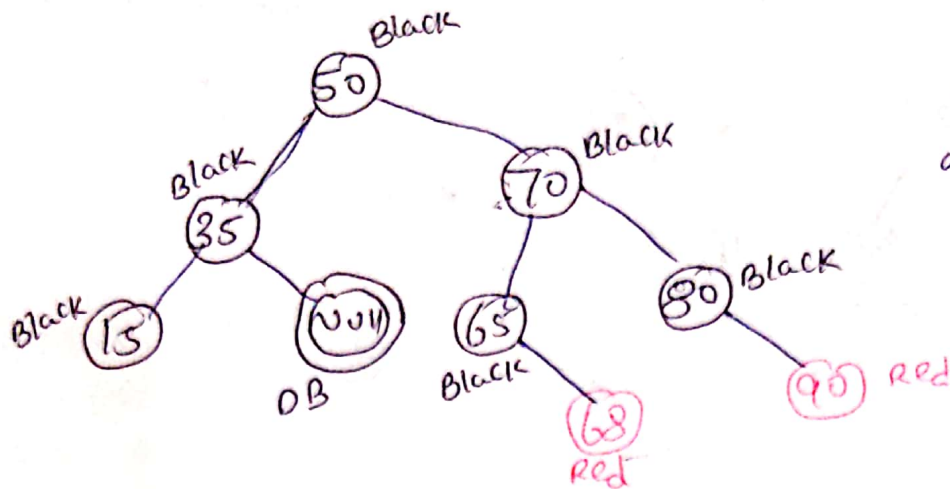
Parent of ~~DB~~
add DB to its
Parent. Since
Parent is Red (65)
change to Black.
and make sibling
to Red. (68)

Now, the above tree is Red-Black tree
after deleting (55)

4

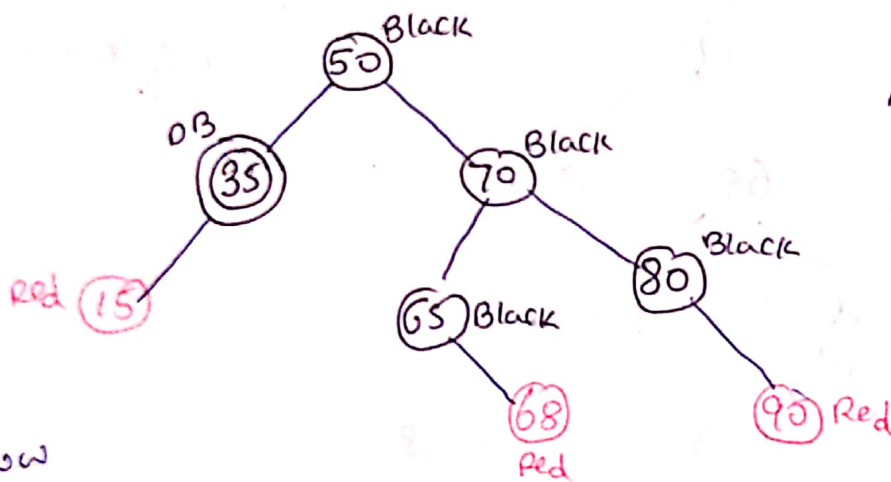
Delete 20

Delete 20 as per BST, and add Double Black



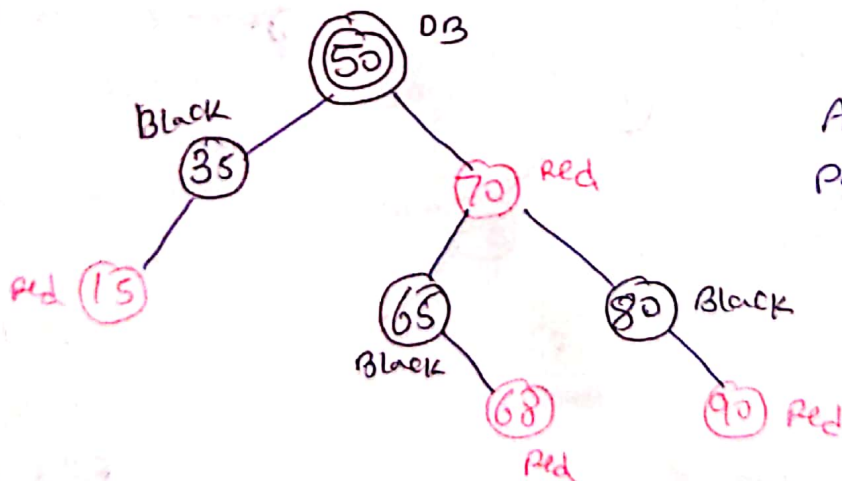
20 is replaced with 35 and 20 is deleted by adding DB

Apply Case 3, and Case 3.2.



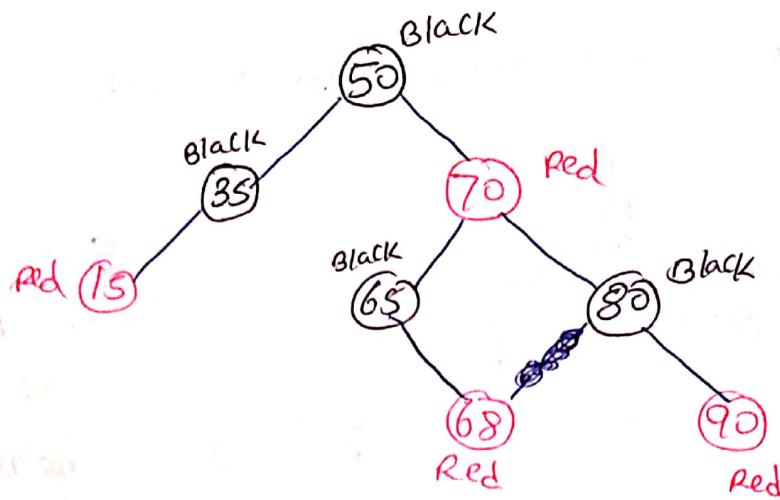
Add Black to its Parent then 35 will become DB and sibling becomes Red

now Apply Case 3.1 & 3.2 again.



Add Black to its Parent then 50 will become DB and sibling becomes Red

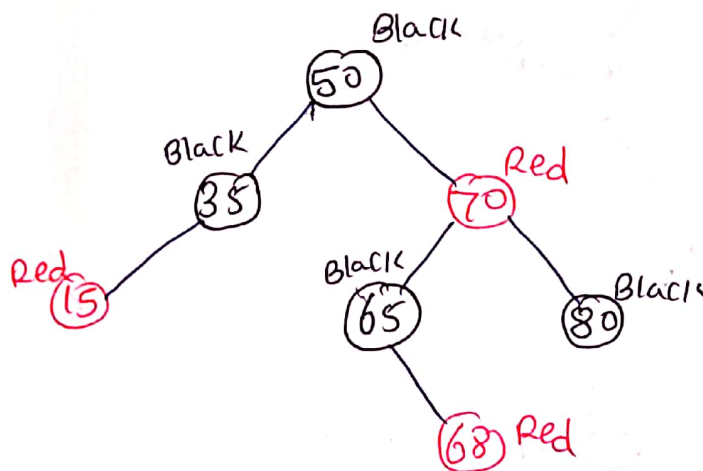
now According to Case 2, the Root is DB so simply remove it the updated tree after deleting 20 is as follows



delete node 90

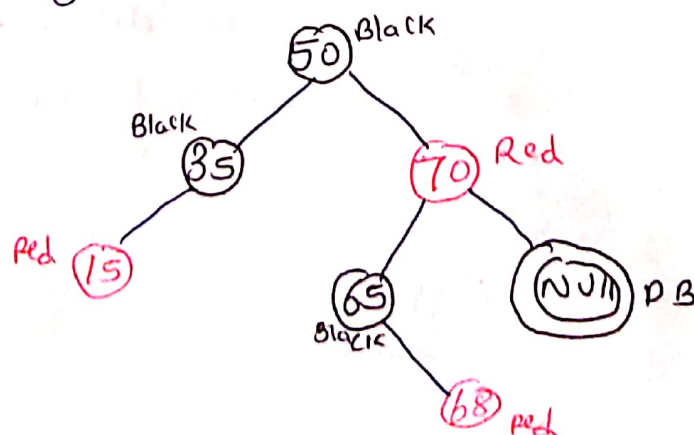
Apply

Apply BST deletion, and according to case 1
if node to be deleted is Red simply delete it

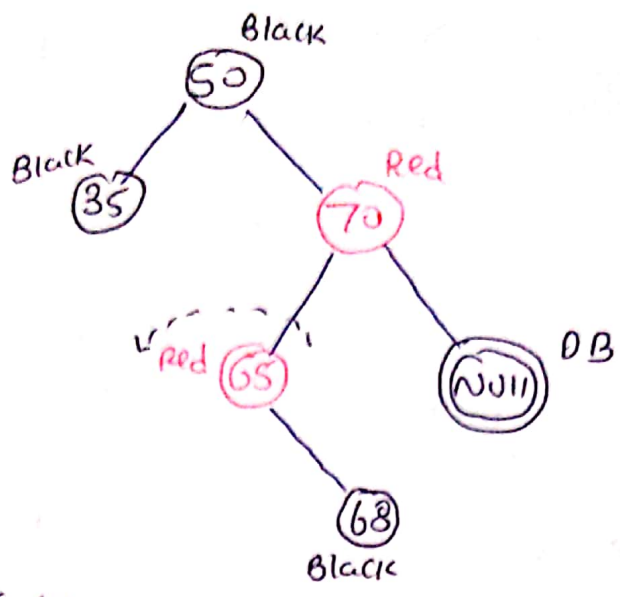


delete node 80

Delete node using BST method. 80 is Black node
so replace 80 with null node and it becomes double Black (DB)

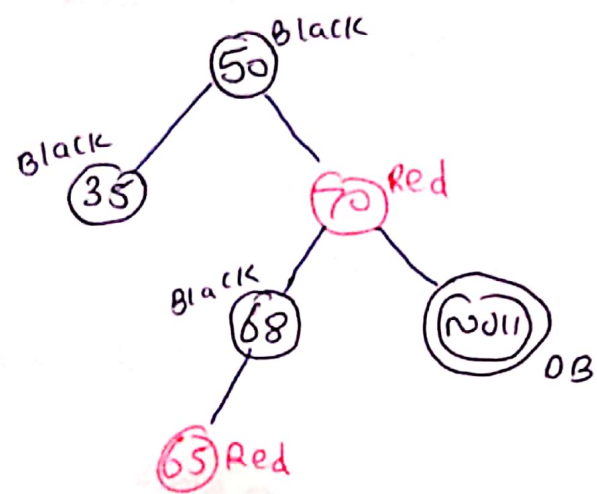


Now apply case 5. According to case 5 rule

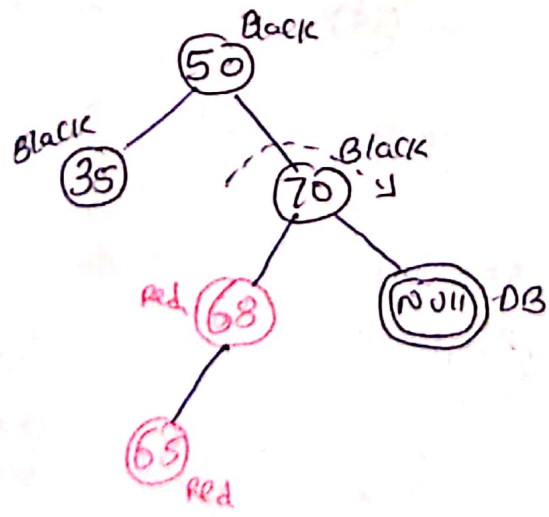


Swap the colors of DB's sibling and sibling child who is near to DB

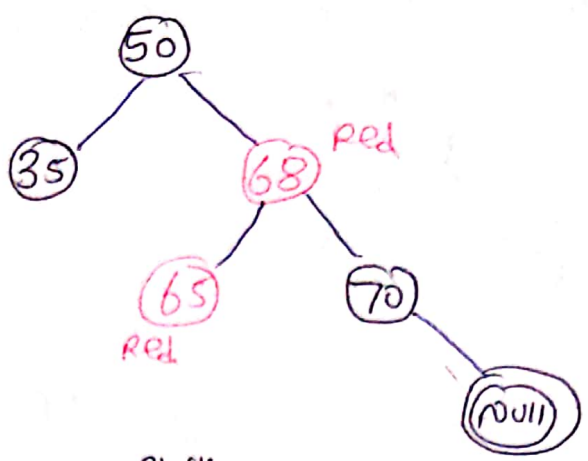
Rotate sibling in opposite direction of DB



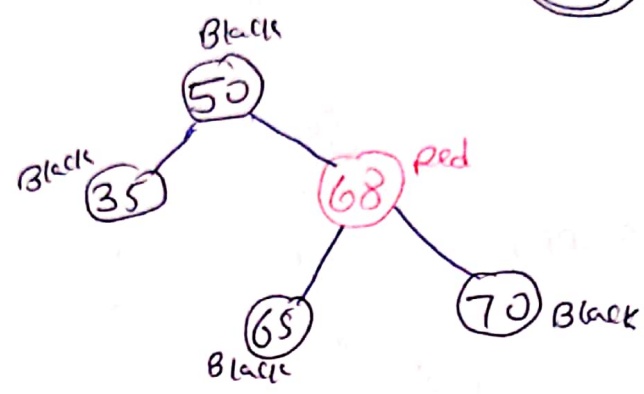
Again apply case 6 and case 2



Swap color of parent and sibling of DB



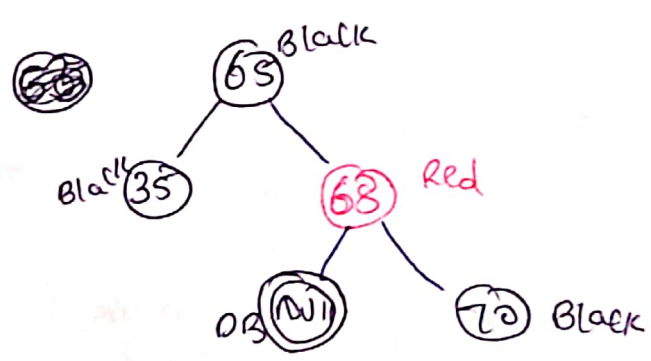
Rotate Parent
in DB Direction



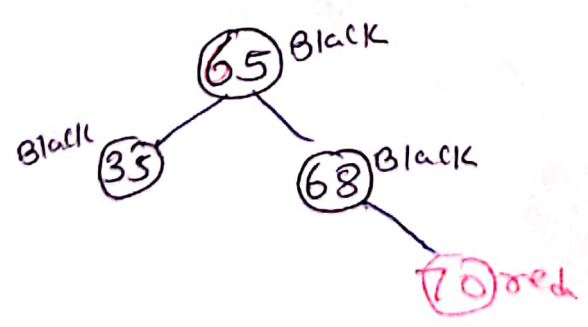
Remove DB and
change the color of
Red child(65) to
Black

Remove 50

Apply as per BST delete so node 65 is replaced
in place of 50 and node 65 will get delete and becomes
DB.



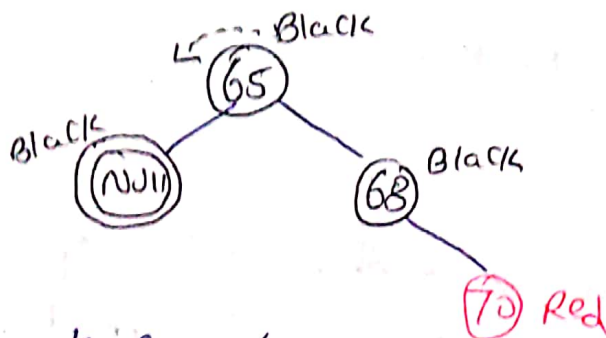
Apply case 3 and 3-1



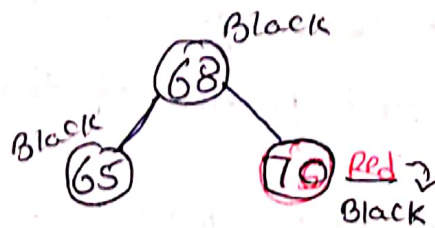
Remove DB and
Parent of DB becomes
Black as it is in Red
color and sibling becomes
Red

Remove 35

Apply BST to remove 35, After removing 35 it is replaced with null and becomes OB



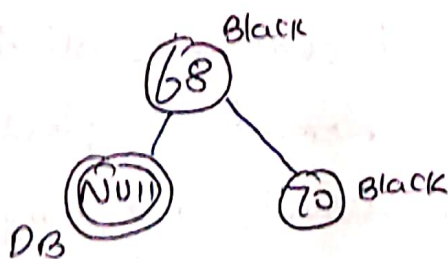
According to case 6 swap color of Parent and Sibling
 ∴ Both Parent and Sibling have same color so no change
 in here then rotate Parent in OB's direction



remove OB and change
 the color of Red child to
 Black

Remove 65

Also Apply BST rule to remove 65 so 65 is replaced
 with null and becomes OB



Apply case 3 and 3.2



Parent
 becomes OB
 and Sibling
 becomes Red

According to case 2 Root is OB so just remove it



Remove (68)

APPLY BST to remove (68) 70 is replaced in 68 and 70 is removed according to case ①

(70)^{Black}

Splay Tree:-

Splay tree is another variant of a binary search tree. In a splay tree, recently accessed element is placed at the root of the tree. A splay tree is defined as follow...

Splay Tree is a self-adjusted Binary Search Tree in which every operation on element rearranges the tree so that the element is placed at the root position of the tree.

In a splay tree, every operation is performed at the root of the tree. All the operations in splay tree are involved with a common operation called "Splaying".

Splaying an element, is the process of bringing it to the root position by performing suitable rotation operations.

In a splay tree, splaying an element rearranges all the element in the tree so that splayed element is placed at the root of the tree. By splaying elements we bring more frequently used elements closer to the root of the tree.

So that any operation on those elements is performed quickly. That means the splaying operation automatically

brings more frequently used element. Every operation on splay tree performs the Splay operation. For example, the insertion operation first inserts the new element using the binary search tree insertion process, then the newly inserted element is splayed so that it is placed at the root of the tree. The search operation in a splay tree is nothing but searching the element using binary search process. In splay tree, to splay any element we use the following notation operations...

Rotation in Splay Tree:

- * Zig Rotation
- * Zag Rotation
- * Zig - Zig Rotation
- * Zag - Zag Rotation
- * Zig - Zag Rotation
- * Zag - Zig Rotation

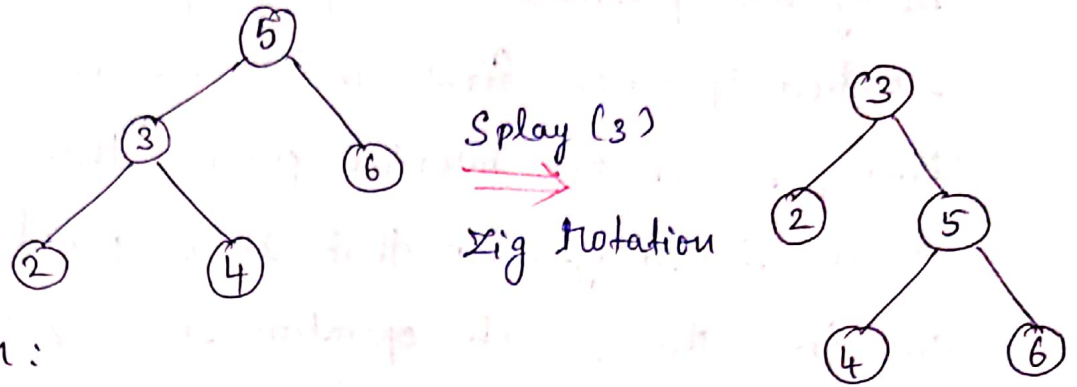
Example :

Zig Rotation

The Zig Rotation in splay tree is similar to the single right rotation in AVL Tree rotations. In Zig rotation, every node moves one position to the right from its current position.

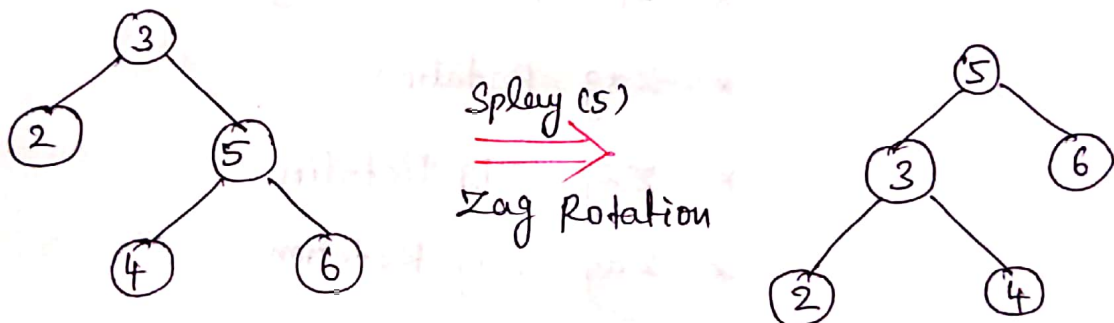
Consider the following example...

(2)



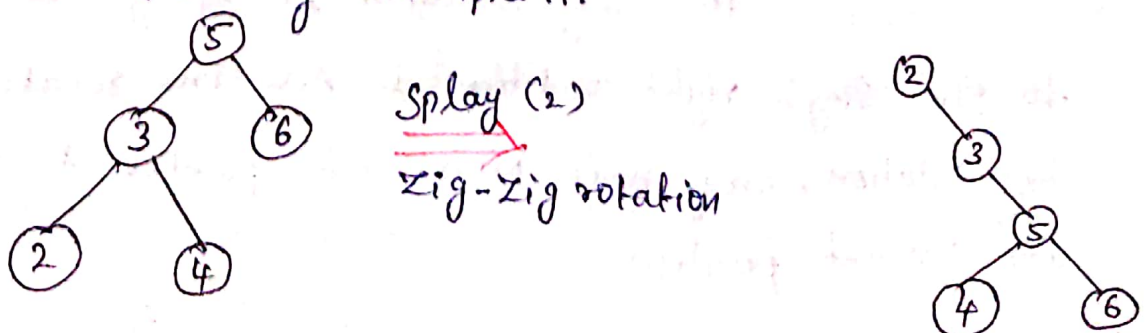
Zag Rotation:

The Zag Rotation in splay tree is similar to the single left rotation in AVL Tree rotations. In Zag rotation, every node moves one position to the left from its current position. Consider the following example...



Zig-Zig Rotation:

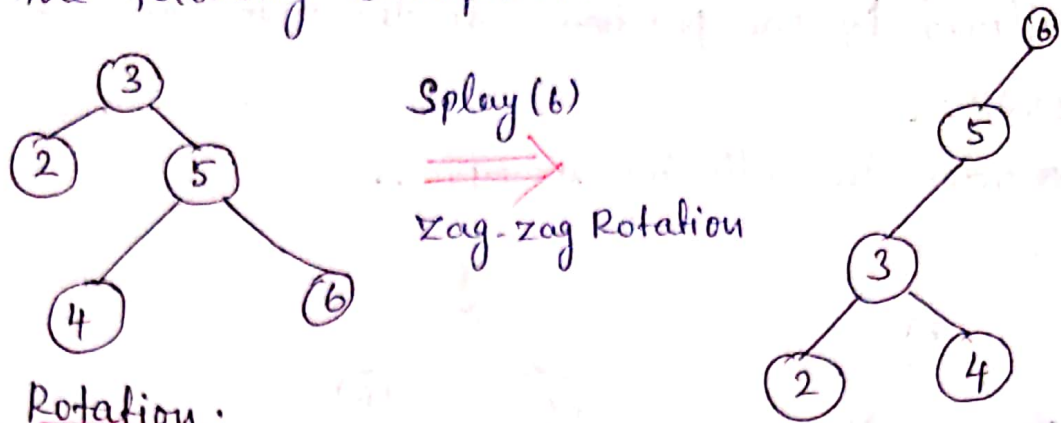
The Zig-Zig Rotation in splay tree is a double Zig rotation. In Zig-Zig Rotation, every node moves two positions to the right from its current position. Consider the following example...



Zag-Zag Rotation:

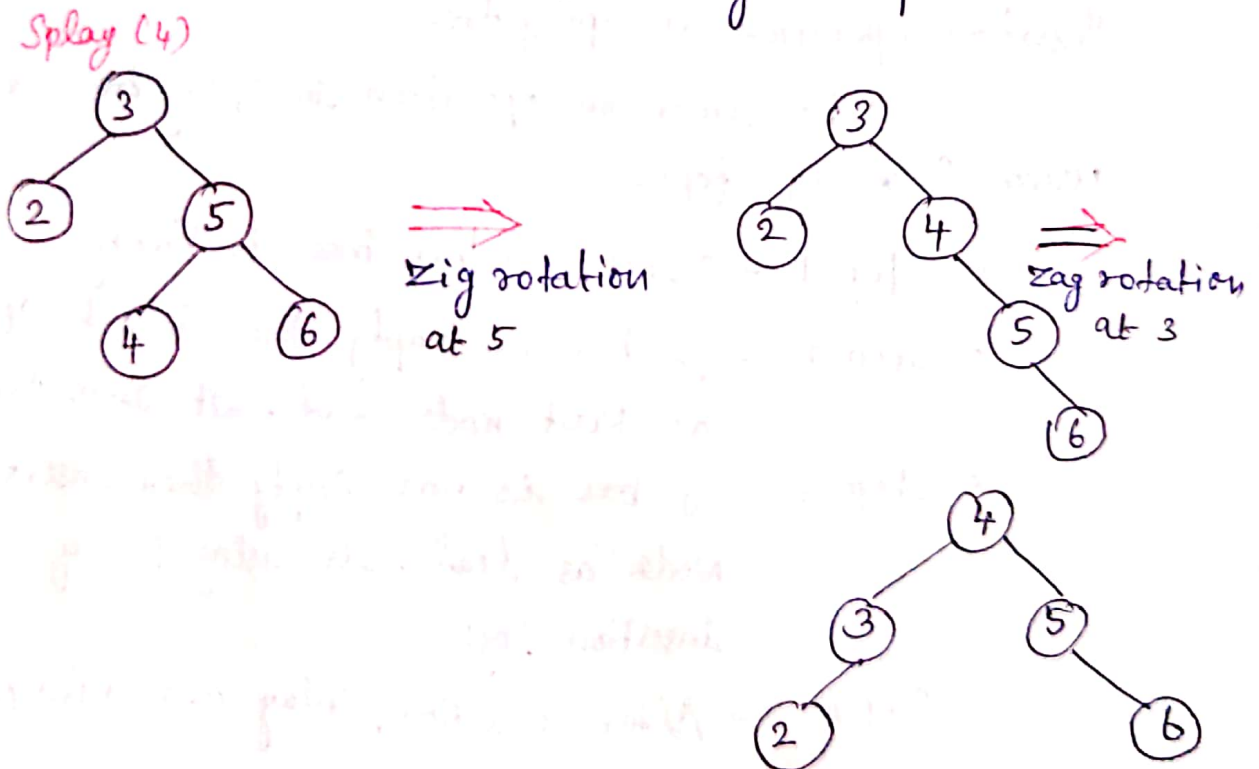
22

The Zag-Zag Rotation in splay tree is a double zag rotation. In Zag-Zag rotation, every node moves two position to the left from its current position. Consider the following example...



Zig-Zag Rotation:

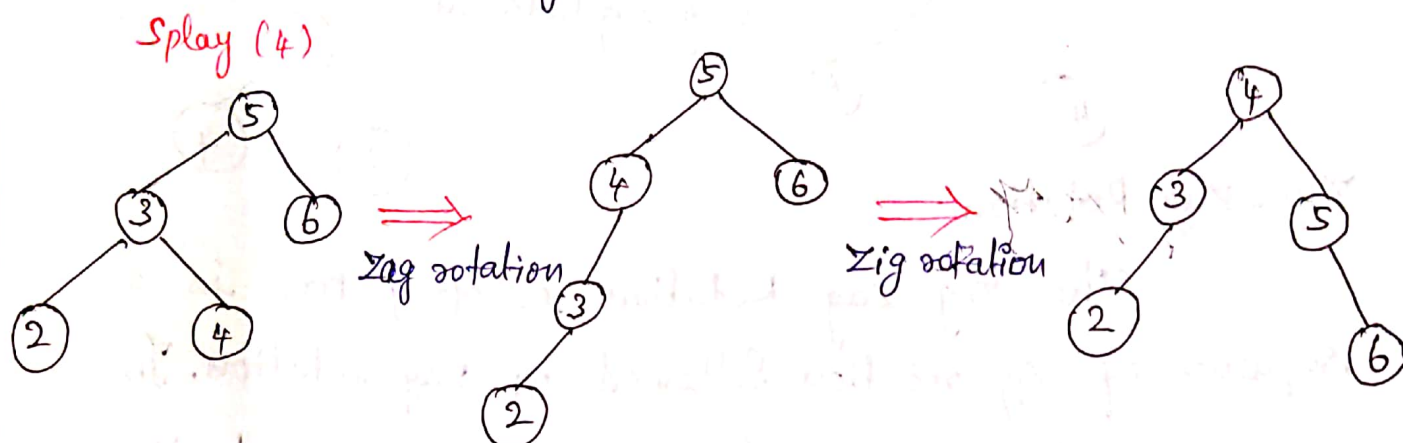
The Zig-Zag Rotation in splay tree is a sequence of zig rotation followed by zag rotation. In Zig-Zag rotation, every node moves one position to the right followed by one position to the left from its current position. Consider the following example...



Zag-Zig Rotation :

The Zag-Zig Rotation in splay tree is a sequence of zag rotation followed by zig rotation. In Zag-Zig rotation, every node moves one position to the left followed by one position to the right from its current position.

Consider the following example...



Every Splay tree must be a binary search tree but it is need not to be balanced tree.

Insertion Operation in splay tree

The insertion operation in Splay tree is performed using following steps...

- * Step 1 - Check whether tree is Empty.
- * Step 2 - If tree is Empty then insert the newnode as Root node and exit from the operation.
- * Step 3 - If tree is not Empty then insert the new Node as leaf node using Binary Search tree insertion logic.
- * Step 4 - After insertion, Splay the new Node.

Algorithm:-

Insert(T, n)

$kemp = T \rightarrow root$

$y = null$

while $kemp \neq null$

$y = kemp$

if $n \rightarrow data < kemp \rightarrow data$

$kemp = kemp \rightarrow left$

else

$kemp = kemp \rightarrow right$

$n \rightarrow parent = y$

if $y == null$

$T \rightarrow root = n$

else if $n \rightarrow data < y \rightarrow data$

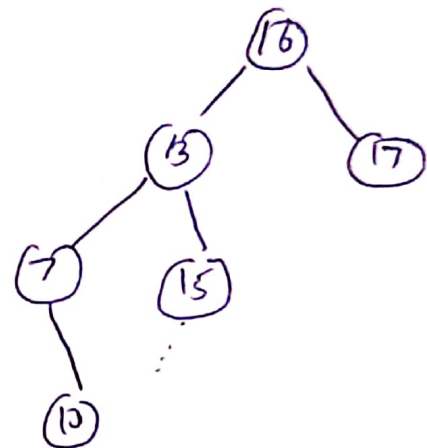
$y \rightarrow left = n$

else

$y \rightarrow right = n$

Splay(T, n)

Insert (44)



Algorithm:

(24)

Splay (T, n)

while $n \rightarrow \text{parent} \neq \text{NULL}$

if $\left\{ \begin{array}{l} n \rightarrow \text{parent} == T \rightarrow \text{root} \\ \text{if } n == n \rightarrow \text{parent} \rightarrow \text{left} \\ \text{right-rotation}(T, n \rightarrow \text{parent}) \\ \text{else left-rotation}(T, n \rightarrow \text{parent}) \end{array} \right.$

else

$p = n \rightarrow \text{parent}$

$g = p \rightarrow \text{parent}$

if $n == n \rightarrow \text{parent} \rightarrow \text{left} \ \&\& \ p == p \rightarrow \text{parent} \rightarrow \text{left}$
right-rotation(T, g), right-rotation(T, p)

else if $n == n \rightarrow \text{parent} \rightarrow \text{right} \ \&\& \ p == p \rightarrow \text{parent} \rightarrow \text{right}$
left-rotation(T, g), left-rotation(T, p)

else if $n == n \rightarrow \text{parent} \rightarrow \text{left} \ \&\& \ p == p \rightarrow \text{parent} \rightarrow \text{right}$
right-rotation(T, p); left-rotation(T, g)

else left-rotation(T, p), right-rotation(T, g)

right-rotation(T, x)

$y = x \rightarrow \text{left}$

$x \rightarrow \text{left} = y \rightarrow \text{right}$

$y \rightarrow \text{right} = x$

return y

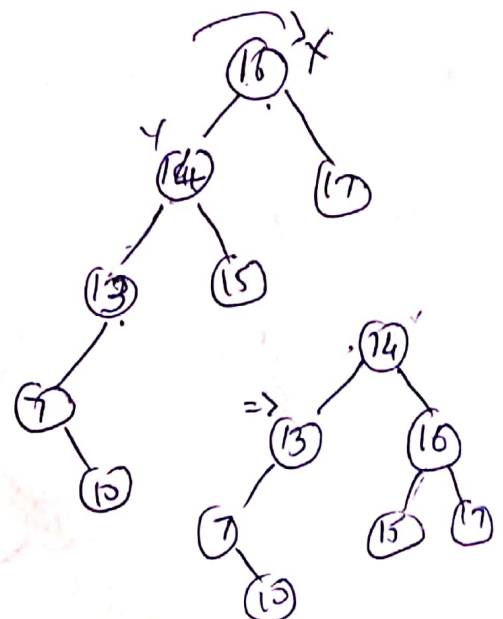
left rotation(T, x)

$y = x \rightarrow \text{right}$

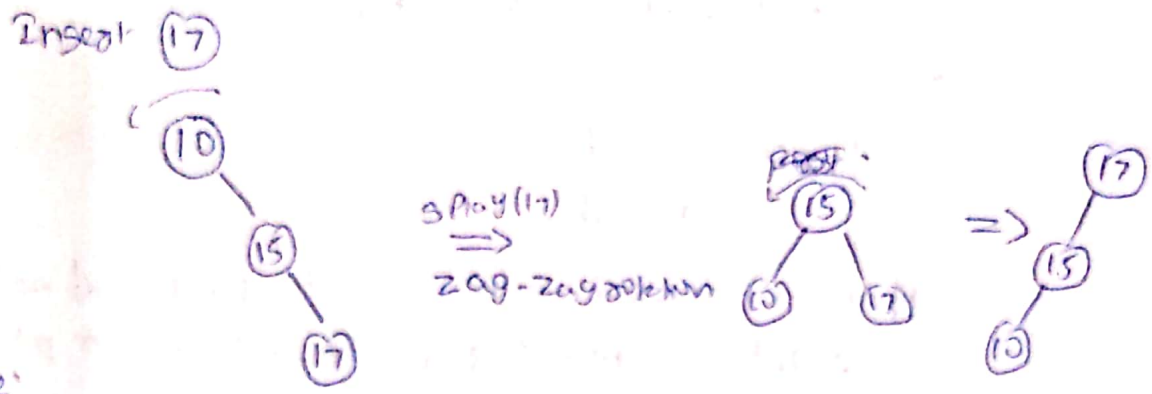
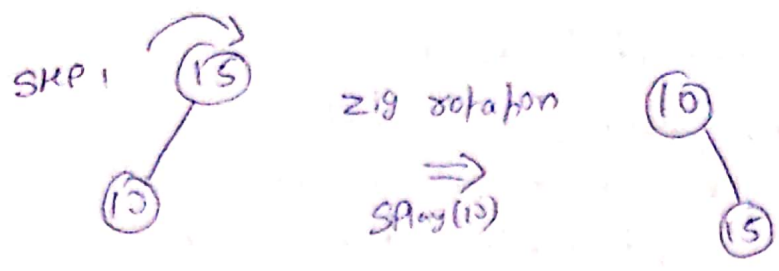
$x \rightarrow \text{right} = y \rightarrow \text{left}$

$y \rightarrow \text{left} = x$

return y

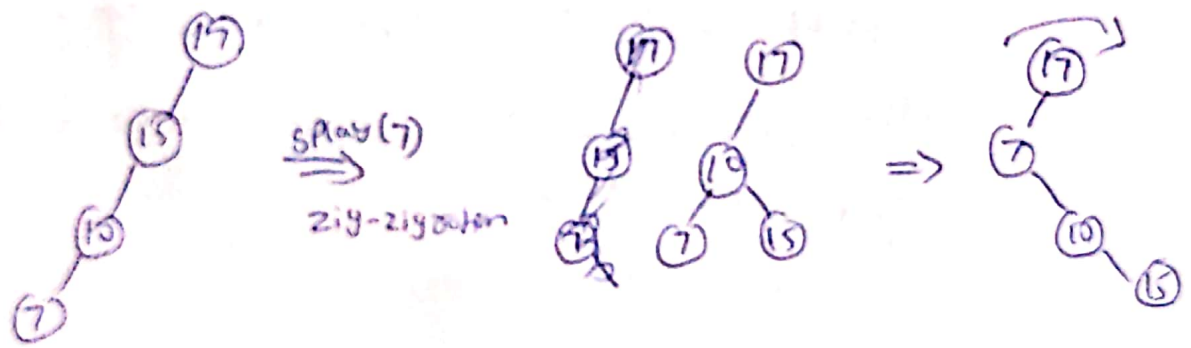


Insert 15, 10, 17, 7, 13, 16



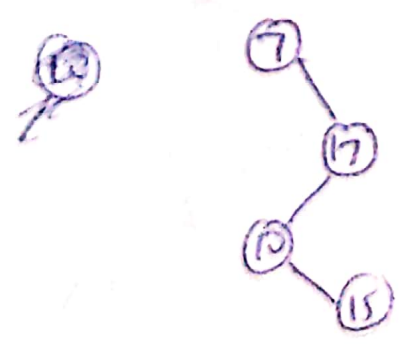
Note: First rotation on Grand Parent and then on Parent node

Insert 7

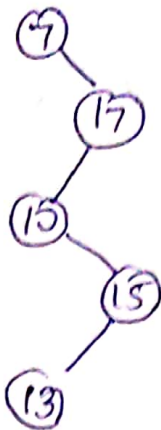


First rotation on Grand Parent and then on Parent node

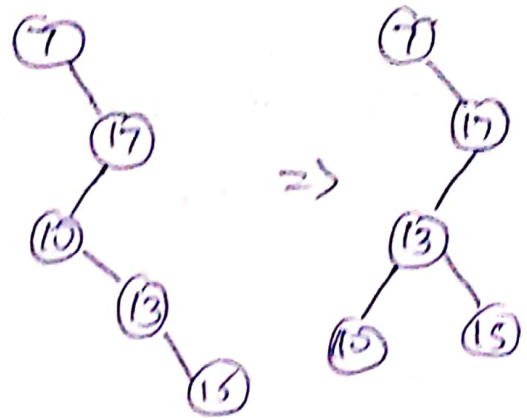
Still 7 is not root so we need to apply zig-zig rotation



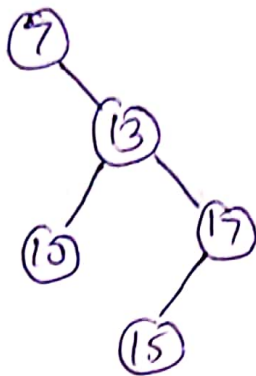
Insert (13)



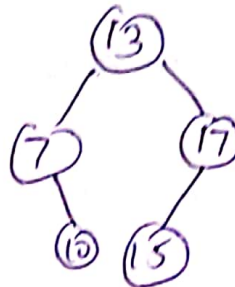
splay(13)
zig-zag rotation



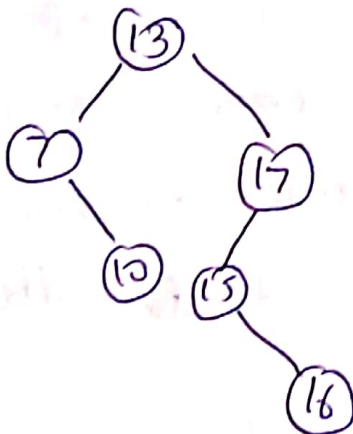
still (13) is not a root so apply again zig-zag rotation



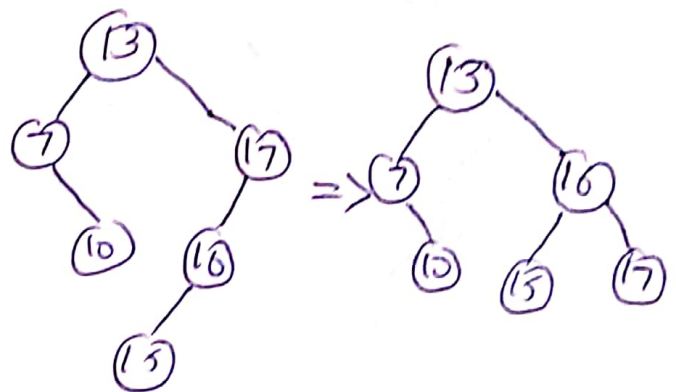
zig-zag ⇒



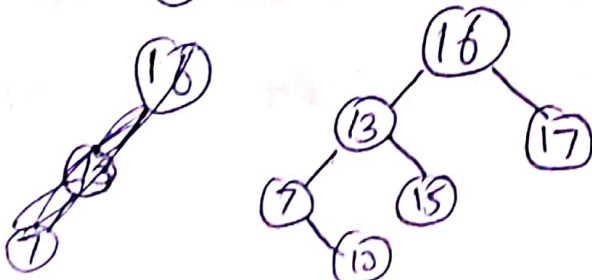
Insert (16)



splay(16)
⇒
Zag-zig rotation



still (16) is not a root so apply zag rotation



Deletion operation on splay tree:

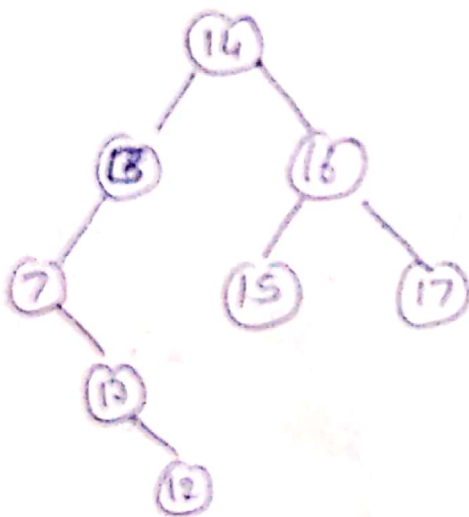
There are two types of deletions in the splay trees.

- Bottom-up splaying
- Top-Down splaying

Bottom-up splaying:

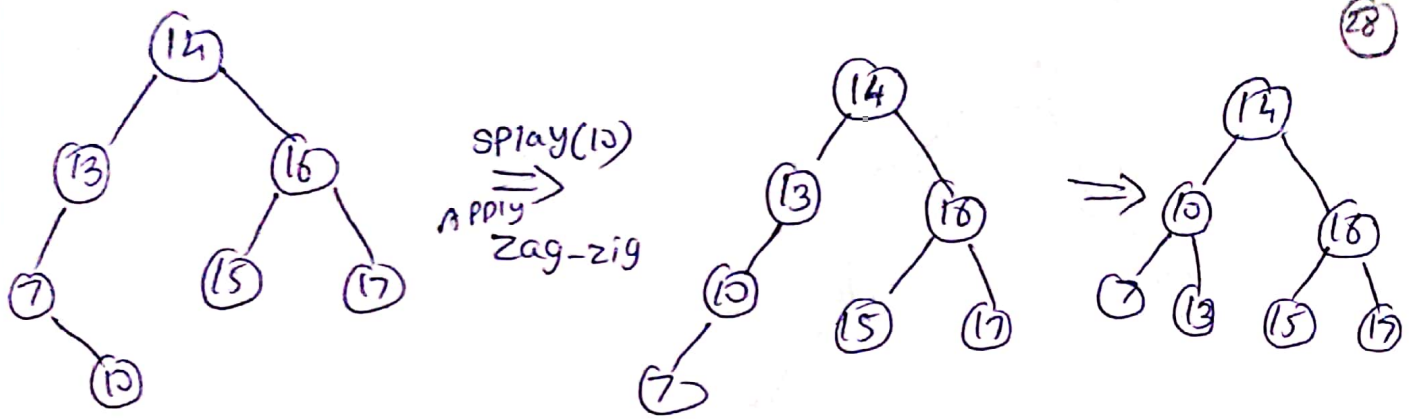
In Bottom-up splaying first we delete the element from the tree and then we perform the splaying on the deleted node.

Suppose we want to delete 12 from the tree shown below.

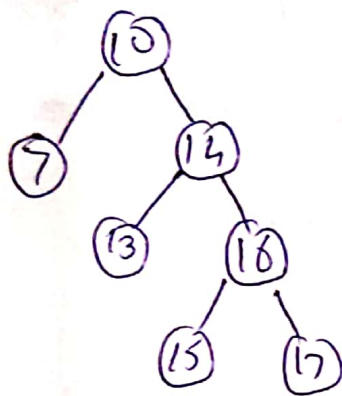


First, we simply perform the standard BST deletion operation to delete 12 element. As 12 is a leaf node, we simply delete the node from the tree.

After deleting 12, we need to splay on the parent of deleted node i.e. 10. We now perform $\text{splay}(10)$ on above tree.



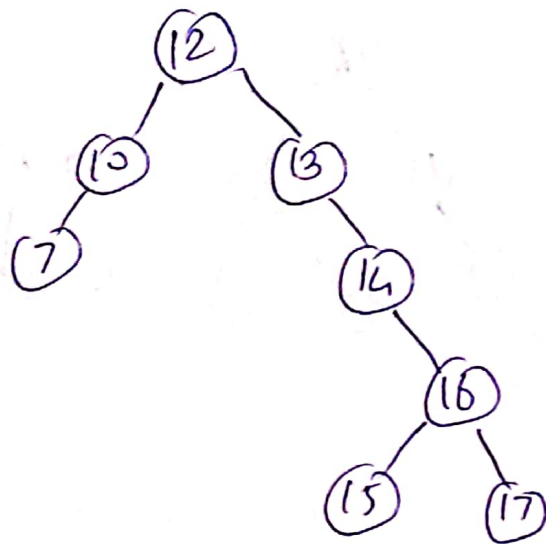
Now apply zig rotation $\because$ 10 is still not a root



Top-down Splaying:-

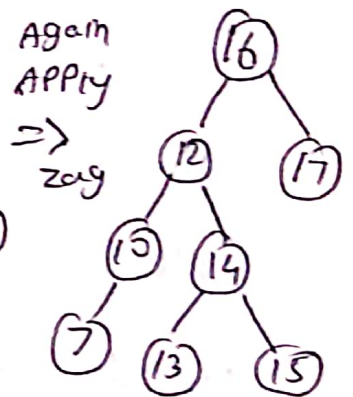
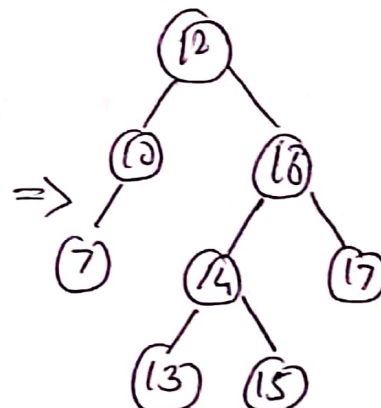
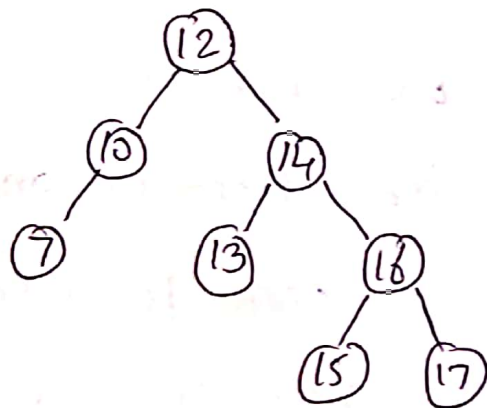
In top-down splaying we first perform the splaying which the deletion is to be performed and then delete the node from the tree. Once the element is deleted, we will perform join operation.

Ex: Suppose we want to delete 16 from the tree given below

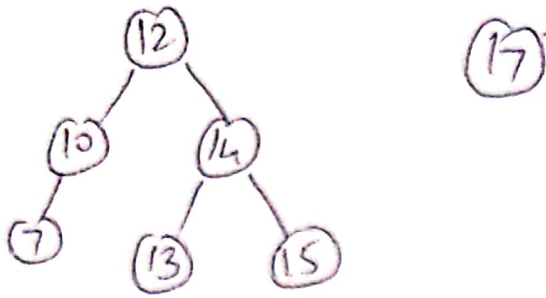


In top-down splaying, first we perform splaying on which the deletion is to be performed and then on the node 16. The node 16 has make node 16 to the root

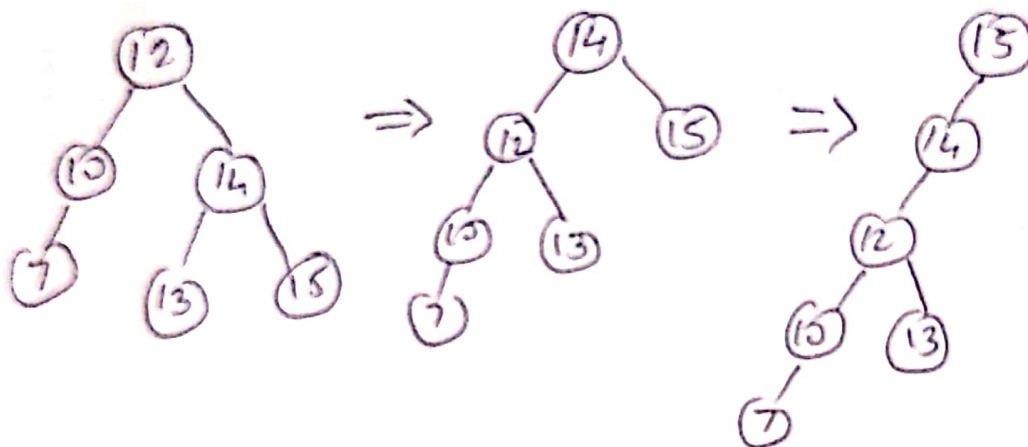
Apply zig-zag rotation



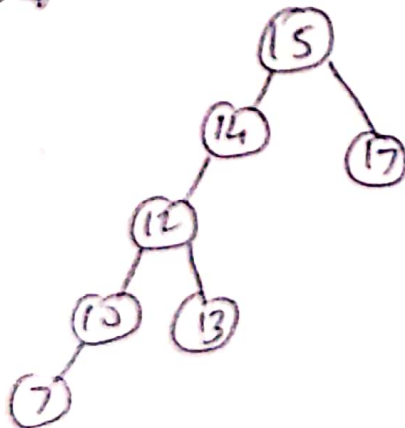
Once the node 16 becomes root node, we will delete the node 16 and we will get two different tree i.e. left subtree and right subtree as shown below



The first step is to find the maximum element in the left subtree. In the left subtree, the maximum element is 15, and then we need to perform splay operation on 15.



Now attack right subtree 17 if it's there we get



Deletion:

(31)

Algorithm:

```
if (root == NULL)
    return NULL
Splay (root, data)
if data != root → data
element is not present
if root → left == NULL
    root = root → right
else
    temp = root
    Splay (root → left, data)
    root → right = root → right
    free(temp)
    return root
```

Application of splay tree

- It is fastest type of a BST for various practical applications
- It is used in windows NT, and GCC compilers.
- It provides better performance as the frequently accessed nodes will move nearer to the root node.
- Splay trees are typically used in the implementation of caches, memory allocators, garbage collectors, data compression, etc.

Hash Tables:

Introduction:

We've seen searches that allow you to look through data in $O(n)$ time, and searches that allow you to look through data in $O(\log n)$ time, but imagine a way to find exactly what you want in $O(1)$ time. Think it's not possible? Think again! Hash tables allow the storage and retrieval of data in an average time of $O(1)$. At its most basic level, a hash table data structure is just an array. Data is stored into this array at specific indices designated by a hash function. A hash function is a mapping between the set of input data and a set of integers.

With hash tables, there always exists the possibility that two data elements will hash to the same integer value. When this happens, a collision results (two data members try to occupy the same place in the hashtable array), and methods have been devised to deal with such situations. In this guide, we will cover two methods, linear probing and separate chaining, focusing on the latter.

A hash table is made up of two parts: an array (the actual table where the data to be searched is stored) and a mapping function, known as a hash function. The hash function is a mapping from the input space to the integer space that defines the indices of the array. In other words, the hash function provides a way for assigning numbers to the input data such that the data can then be stored at the array index corresponding to the assigned number.

Let's take a simple example. First, we start with a hash table array of strings (we'll use strings as the data being stored and searched in this example). Let's say the hash table size is 12:

Hash Table(strings)	
0	(null)
1	(null)
2	(null)
3	(null)
4	(null)
5	(null)
6	(null)
7	(null)
8	(null)
9	(null)
10	(null)
11	(null)

Next we need a hash function. There are many possible ways to construct a hash function. We'll discuss these possibilities more in the next section. For now, let's assume a simple hash function that takes a string as input. The returned hash value will be the sum of the ASCII characters that

make up the string mod the size of the table:

```
int hash(char *str, int table_size) { int sum; /* Make sure a valid string passed in */ if
(str==NULL) return -1; /*
Sum up all the characters in the string */ for( ; *str; str++) sum += *str; /* Return the
sum mod the table size */ return sum % table_size; }
```

We run "Steve" through the hash function, and find that hash("Steve",12) yields 3:

0	(null)
1	(null)
2	(null)
3	"Steve"
4	(null)
5	(null)
6	(null)
7	(null)
8	(null)
9	(null)
10	(null)
11	(null)

The hash table after inserting "Steve"

Let's try another string: "Spark". We run the string through the hash function and find that hash("Spark",12) yields 6. Fine. We insert it into the hash table:

0	(null)
1	(null)
2	(null)
3	"Steve"
4	(null)
5	(null)
6	"Spark"
7	(null)
8	(null)
9	(null)
10	(null)
11	(null)

The hash table after inserting "Spark"

Let's try another: "Notes". We run "Notes" through the hash function and find that hash("Notes",12) is 3.Ok. We insert it into the hash table:

Hash Table(strings)	
0	(null)
1	(null)
2	(null)
3	"Steve" "Notes"
4	(null)
5	(null)
6	"Spark"
7	(null)
8	(null)
9	(null)
10	(null)
11	(null)

A hash table collision

What happened? A hash function doesn't guarantee that every input will map to a different output. There is always the chance that two inputs will hash to the same output. This indicates that both elements should be inserted at the same place in the array, and this is impossible. This phenomenon is known as a collision.

There are many algorithms for dealing with collisions, such as linear probing and separate chaining. While each of the methods has its advantages, we will only discuss separate chaining here.

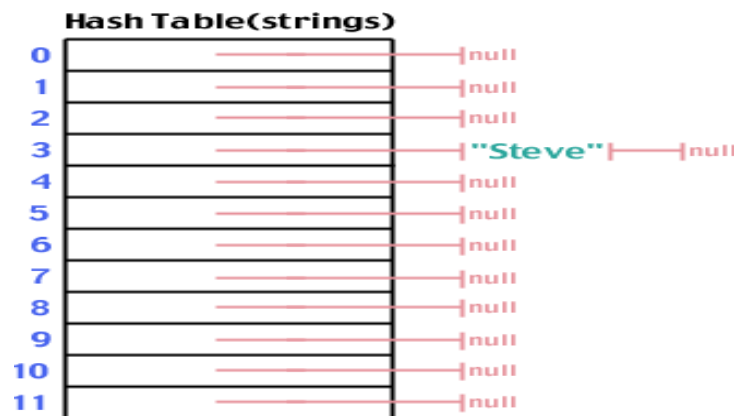
Separate chaining requires a slight modification to the data structure. Instead of storing the data elements right into the array, they are stored in linked lists. Each slot in the array then points to one of these linked lists. When an element hashes to a value, it is added to the linked list at that index in the array. Because a linked list has no limit on length, collisions are no longer a problem. If more than one element hashes to the same value, then both are stored in that linked list.

Let's look at the above example again, this time with our modified data structure:



Modified table for separate chaining

Again, let's try adding "Steve" which hashes to 3:



After adding "Steve" to the table and "Spark" which hashes to 6:

Problem : How does a hash table allow for $O(1)$ searching? What is the worst case efficiency of a look up in a hash table using separate chaining?

A hash table uses hash functions to compute an integer value for data. This integer value can then be used as an index into an array, giving us a constant time access to the requested data. However, using separate chaining, we won't always achieve the best and average case efficiency of $O(1)$. If we have too small a hash table for the data set size and/or a bad hash function, elements can start to build in one index in the array. Theoretically, all n element could end up in the same linked list. Therefore, to do a search in the worst case is equivalent to looking up a data element in a linked list, something we already know to be $O(n)$ time. However, with a good hash function and a well created hash table, the chances of this happening are, for all intents and purposes, ignorable.

Problem : The bigger the ratio between the size of the hash table and the number of data elements, the less chance there is for collision. What is a drawback to making the hash table big enough so the chances of collision is ignorable?

Wasted memory space

Problem : How could a linked list and a hash table be combined to allow someone to run through the list from item to item while still maintaining the ability to access an individual element in $O(1)$ time?

Hash Functions

As mentioned briefly in the previous section, there are multiple ways for constructing a hash function. Remember that hash function takes the data as input (often a string), and returns an integer in the range of possible indices into the hash table. Every hash function must do that, including the bad ones. So what makes for a good hash function?

Characteristics of a Good Hash Function

There are four main characteristics of a good hash function:

- 1) The hash value is fully determined by the data being hashed.
- 2) The hash function uses all the input data.
- 3) The hash function "uniformly" distributes the data across the entire set of possible hash values.
- 4) The hash function generates very different hash values for similar strings. Let's examine why each of these is important:

Rule 1: If something else besides the input data is used to determine the hash, then the hash value is not as dependent upon the input data, thus allowing for a worse distribution of the hash values.

Rule 2: If the hash function doesn't use all the data would cause an inappropriate number of similar hash values resulting in too many collisions.

Rule 3: If the hash function does not uniformly distribute the data across the entire set of possible hash values, a large number of collisions will result, cutting down on the efficiency of the hash table.

Rule 4: In real world applications, many data sets contain very similar data elements.

Hash Table is a data structure which stores data in an associative manner. In a hash table, data is stored in an array format, where each data value has its own unique index value. Access of data becomes very fast if we know the index of the desired data.

Thus, it becomes a data structure in which insertion and search operations are very fast irrespective of the size of the data. Hash Table uses an array as a storage medium and uses hash technique to generate an index where an element is to be inserted or is to be located from.

Hashing

Hashing is a technique to convert a range of key values into a range of indexes of an array. We're going to use modulo operator to get a range of key values. Consider an example of hash table of size 20, and the following items are to be stored. Items are in the (key,value) format.

- (1,20)
- (2,70)
- (42,80)
- (4,25)

- (12,44)
- (14,32)
- (17,11)
- (13,78)
- (37,98)

Sr.No.	Key	Hash	Array Index
1	1	$1 \% 20 = 1$	1
2	2	$2 \% 20 = 2$	2
3	42	$42 \% 20 = 2$	2
4	4	$4 \% 20 = 4$	4
5	12	$12 \% 20 = 12$	12
6	14	$14 \% 20 = 14$	14
7	17	$17 \% 20 = 17$	17
8	13	$13 \% 20 = 13$	13
9	37	$37 \% 20 = 17$	17

Linear Probing

As we can see, it may happen that the hashing technique is used to create an already used index of the array. In such a case, we can search the next empty location in the array by looking into the next cell until we find an empty cell. This technique is called linear probing.

Sr.No.	Key	Hash	Array Index	After Linear Probing, Array Index
1	1	$1 \% 20 = 1$	1	1
2	2	$2 \% 20 = 2$	2	2
3	42	$42 \% 20 = 2$	2	3
4	4	$4 \% 20 = 4$	4	4
5	12	$12 \% 20 = 12$	12	12
6	14	$14 \% 20 = 14$	14	14
7	17	$17 \% 20 = 17$	17	17
8	13	$13 \% 20 = 13$	13	13
9	37	$37 \% 20 = 17$	17	18

Open Addressing

Like separate chaining, open addressing is a method for handling collisions. In Open Addressing, all elements are stored in the hash table itself. So at any point, the size of the table must be greater than or equal to the total number of keys (Note that we can increase table size by copying old data if needed).

Insert(k): Keep probing until an empty slot is found. Once an empty slot is found, insert k.

Search(k): Keep probing until slot's key doesn't become equal to k or an empty slot is reached.

Delete(k): **Delete operation is interesting.** If we simply delete a key, then the search may fail. So slots of deleted keys are marked specially as "deleted". The insert can insert an item in a deleted slot, but the search doesn't stop at a deleted slot.

Open Addressing is done in the following ways:

Linear Probing: In linear probing, we linearly probe for next slot. For example, the typical gap between two probes is 1 as seen in the example below. Let **hash(x)** be the slot index computed using a hash function and **S** be the table size

If slot $\text{hash}(x) \% S$ is full, then we try $(\text{hash}(x) + 1) \% S$

If $(\text{hash}(x) + 1) \% S$ is also full, then we try $(\text{hash}(x) + 2) \% S$. If $(\text{hash}(x) + 2) \% S$ is also full, then we try $(\text{hash}(x) + 3) \% S$

Let us consider a simple hash function as "key mod 7" and a sequence of keys as 50, 700, 76, 85, 92, 73, 101.

Challenges in Linear Probing :

1. **Primary Clustering:** One of the problems with linear probing is Primary clustering, many consecutive elements form groups and it starts taking time to find a free slot or to search for an element.
2. **Secondary Clustering:** Secondary clustering is less severe, two records only have the same collision chain (Probe Sequence) if their initial position is the same.

Quadratic Probing We look for i^2 'th slot in i 'th iteration. Let $\text{hash}(x)$ be the slot index computed using hash function.

If slot $\text{hash}(x) \% S$ is full, then we try $(\text{hash}(x) + 1*1) \% S$

If $(\text{hash}(x) + 1*1) \% S$ is also full, then we try $(\text{hash}(x) + 2*2) \% S$

If $(\text{hash}(x) + 2*2) \% S$ is also full, then we try $(\text{hash}(x) + 3*3) \% S$

Double Hashing

We use another hash function $\text{hash}_2(x)$ and look for $i*\text{hash}_2(x)$ slot in i^{th} rotation. let $\text{hash}(x)$ be the slot index computed using hash function.

If slot $\text{hash}(x) \% S$ is full, then we try $(\text{hash}(x) + 1*\text{hash}_2(x)) \% S$

If $(\text{hash}(x) + 1*\text{hash}_2(x)) \% S$ is also full, then we try $(\text{hash}(x) + 2*\text{hash}_2(x)) \% S$

If $(\text{hash}(x) + 2*\text{hash}_2(x)) \% S$ is also full, then we try $(\text{hash}(x) + 3*\text{hash}_2(x)) \% S$

Comparison:

Linear probing has the best cache performance but suffers from clustering. One more advantage of Linear probing is easy to compute. Quadratic probing lies between the two in terms of cache performance and clustering. Double hashing has poor cache performance but no clustering. Double hashing requires more computation time as two hash functions need to be computed.

S.No.	Separate Chaining	Open Addressing
1.	Chaining is Simpler to implement.	Open Addressing requires more computation.
2.	In chaining, Hash table never fills up, we can always add more elements to chain.	In open addressing, table may become full.
3.	Chaining is Less sensitive to the hash function or load factors.	Open addressing requires extra care to avoid clustering and load factor.
4.	Chaining is mostly used when it is unknown how many and how frequently keys may be inserted or deleted.	Open addressing is used when the frequency and number of keys is known.
5.	Cache performance of chaining is not good as keys are stored using linked list.	Open addressing provides better cache performance as everything is stored in the same table.
6.	Wastage of Space (Some Parts of hash table in chaining are never used).	In Open addressing, a slot can be used even if an input doesn't map to it.
7.	Chaining uses extra space for links.	No links in Open addressing

Performance of Open Addressing:

Like Chaining, the performance of hashing can be evaluated under the assumption that each key is equally likely to be hashed to any slot of the table (simple uniform hashing)

Applications of Hashing

- Message Digest.
- Password Verification.
- Data Structures(Programming Languages)
- Compiler Operation.
- Rabin-Karp Algorithm.
- Linking File name and path together

Short Questions:

1. Define splay tree.
2. What is the idea behind splaying?
3. Define hashing function.
4. Define collision
5. What is open addressing?
6. What are the collision resolution methods?
7. Define separate chaining It is an open hashing technique
8. Define Hashing.
9. What do you mean by hash table?
10. Write the importance of hashing
11. What do you mean by collision in hashing?
12. What are the collision resolution methods?
13. What do you mean by open addressing?
14. List the limitations of linear probing.
15. Mention one advantage and disadvantage of using quadratic probing.
16. what are the properties of red black tree?

Long Questions:

1. Explain Operations on Red Black tree.
2. Explain Operations on splay tree
3. List the merits and demerits of splay tree.

4. Explain the properties of red-black trees. Explain the insertion cases.
5. Explain Chaining method and Open Addressing with an example
6. Explain double hashing and Bucket hashing with an example
7. Consider the hash table of size 10. Use linear probing, insert the keys 72, 27, 36, 24, 63, 81, 92, 101 into the table.
8. Explain difference between Linear Probing and Quadratic Probing with an example
9. Explain Rehashing and Extendible hashing with an example